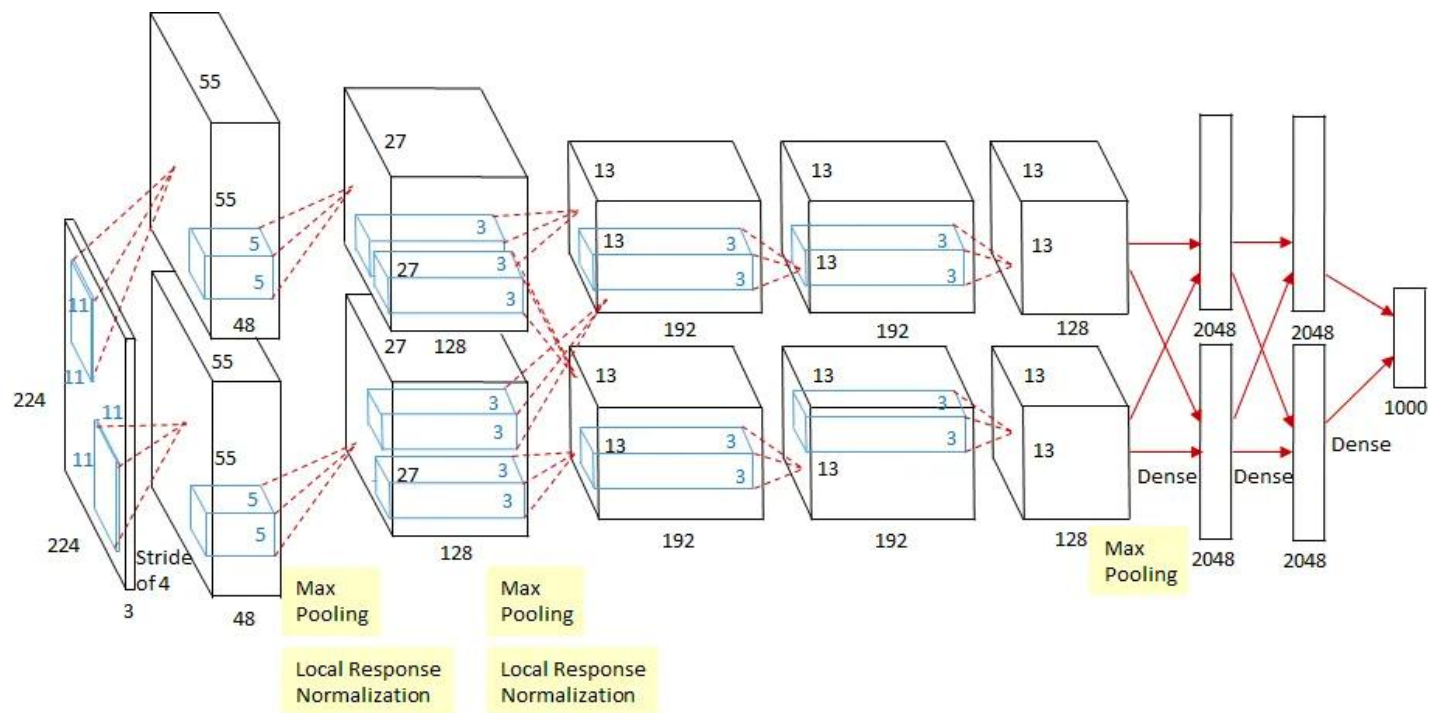


Alexnet

Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," NIPS, 2012



CMES Deep Vision 인턴 박윤준

2025.03.27

| Table of Contents

I. Convolutional Neural Networks (CNN)

II. Why is AlexNet important?

III. AlexNet - Architecture

IV. AlexNet - Key ideas

Chapter |

Convolutional Neural Networks (CNN)

1.1 CNN이 등장한 배경

CNN이 등장한 배경

- 전통적인 인공지능(AI) 및 머신러닝 방법들이 이미지 인식과 같은 복잡한 패턴 인식 문제에서 한계를 보였기 때문

수작업 특징 추출 필요

- 기존 머신러닝 모델은 이미지에서 중요한 특징을 사람이 직접 정의해야 함 (예: SIFT, HOG, Gabor 필터)
- 하지만 이미지 데이터는 복잡해서 모든 경우를 사람이 직접 설계하기 어려움

Fully Connected Layer의 문제점

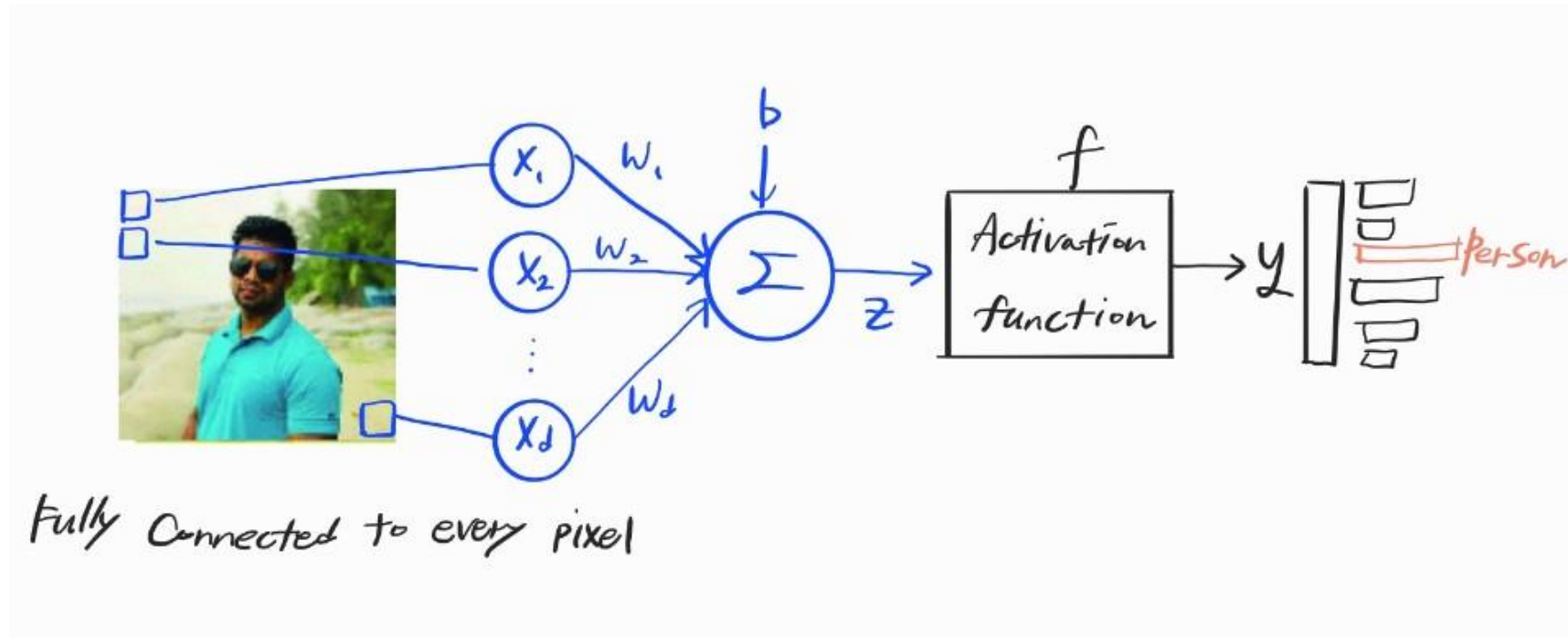
1. 2d 이미지 입력을 Flatten해서 1차원 벡터로 변환한 후 연산을 수행하기 때문에 공간적(Spatial) 위치 정보를 고려하지 않음
 - 즉, 이미지의 픽셀들이 서로 어떤 관계를 맺고 있는지(예: 가까운 픽셀끼리 의미 있는 패턴을 형성하는지)를 반영하지 않고, 모든 픽셀을 독립적인 특징으로 취급
 - fc는 퍼즐처럼 섞어서 학습시키는 거랑 다를 바가 없음
2. FC layer는 각 뉴런이 입력의 모든 픽셀에 연결되어 있기 때문에, 단일 계층의 FC layer는 각 클래스에 대해 하나의 weight map(혹은 template)만 학습할 수 있음
 - 즉, FC layer는 각 클래스에 대해 하나의 template(weight map)을 학습할 뿐, 위치 불변성(spatial invariance)이나 다양한 형태 변형을 일반화하는 능력이 부족하기 때문에 이미지 인식 성능이 떨어지는 문제가 있음 (같은 물체라도 크기나 위치가 다르거나 회전이 되어 있으면 기존 모델이 인식하지 못하는 경우가 많음)
3. 인간처럼 이미지의 특징을 뽑아서 학습해야 좋은데 FC layer는 너무 디테일하게 봄

Weight Map: 네트워크가 학습한 가중치의 공간적 패턴을 의미, Template: 특정 클래스의 대표적인 특징을 하나의 패턴(template)으로 학습

1.1 CNN이 등장한 배경

Fully Connected Layer의 문제점

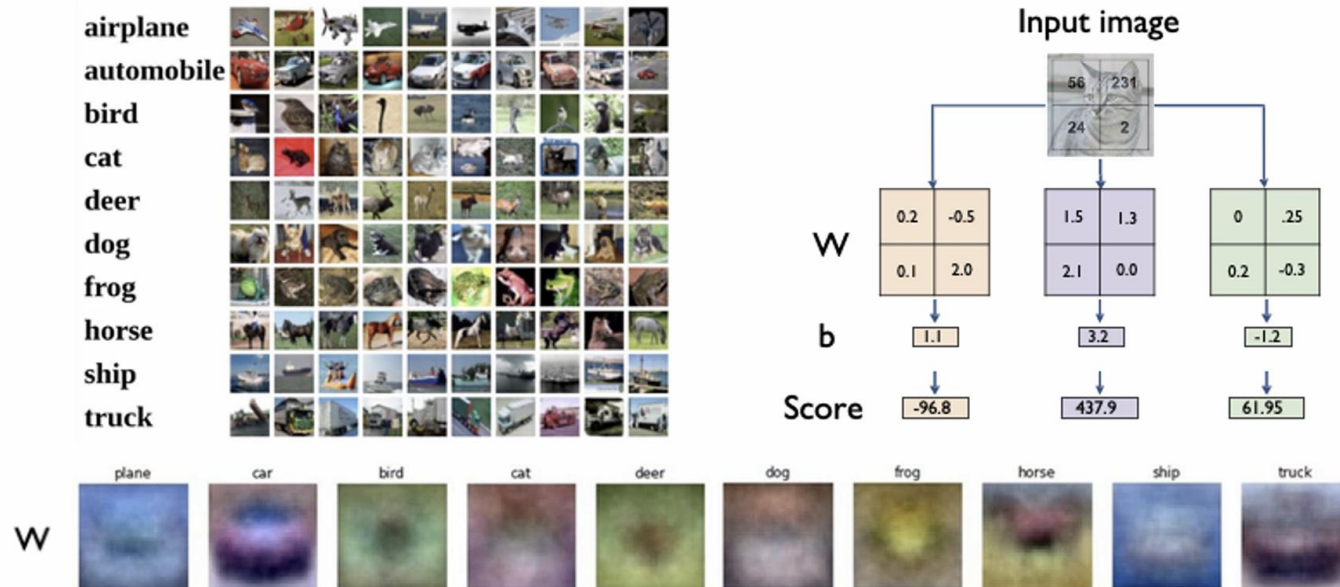
- EX) simple model, perceptron, that takes every pixel of an image as input
- Is this model suitable for solving the image classification problem? No!



1.1 CNN이 등장한 배경

Visualization of single fully connected layer networks

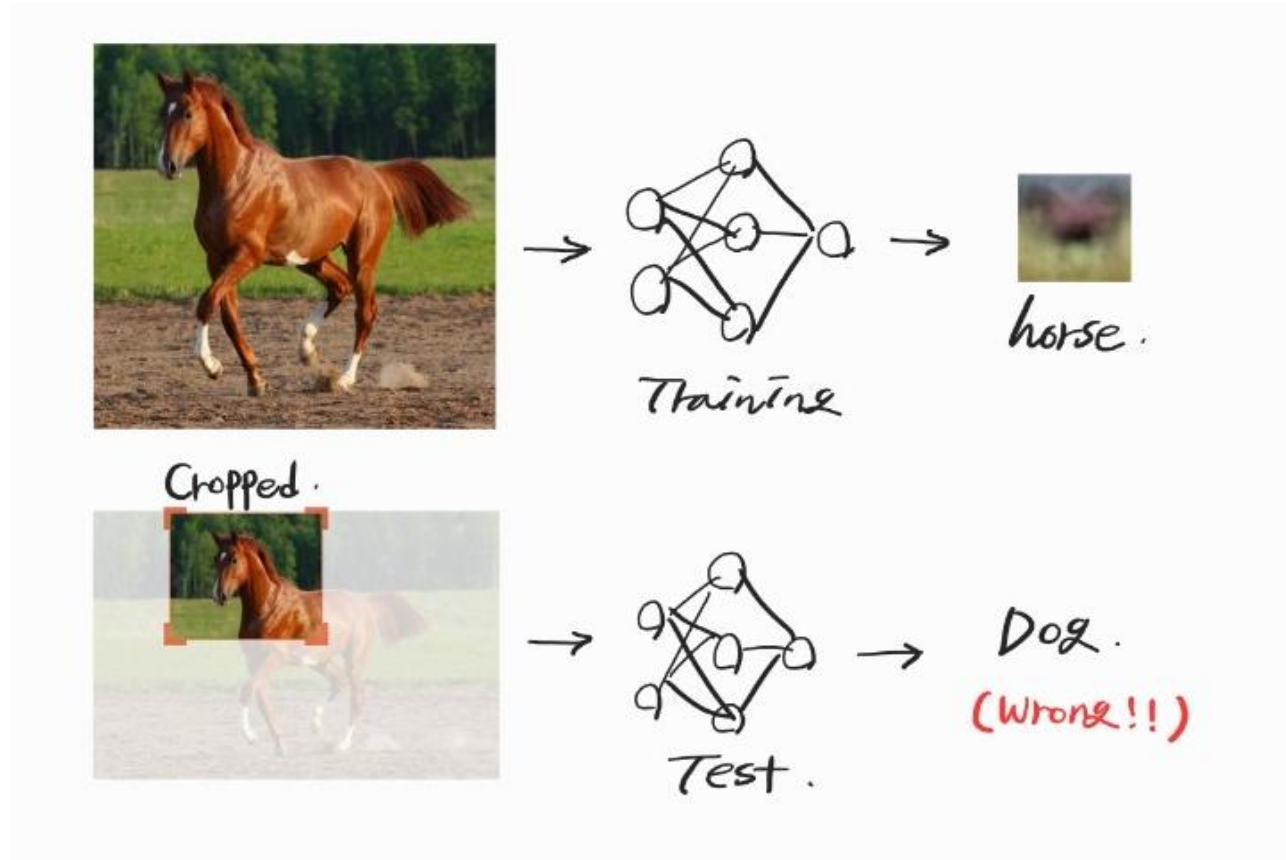
- CIFAR-10 데이터셋을 이용한 분류 모델의 가중치 시각화
- W 부분은 클래스별 대표적인 특징을 보여주는 이미지 템플릿. 즉, 특정 클래스에 대한 "가중치의 평균적인 형태"로 볼 수 있음
- 예를 들어, 자동차(Car) 클래스는 중앙이 밝고 외곽이 어두운 형태를 띄고 있으며, 비행기(Plane)는 전체적으로 푸른색이 많은 형태
- 고양이 예시)
 - 입력 이미지(고양이)가 여러 개의 W, b를 통과하면서 각각의 클래스에 대한 점수를 계산
 - 보라색으로 표시된 W, b에서 나온 점수가 437.9로 가장 높음 → 모델이 이 입력을 "고양이"로 분류.
 - 이 보라색 W, b가 고양이 클래스를 대표하는 템플릿 역할



1.1 CNN이 등장한 배경

Fully Connected Layer의 문제점

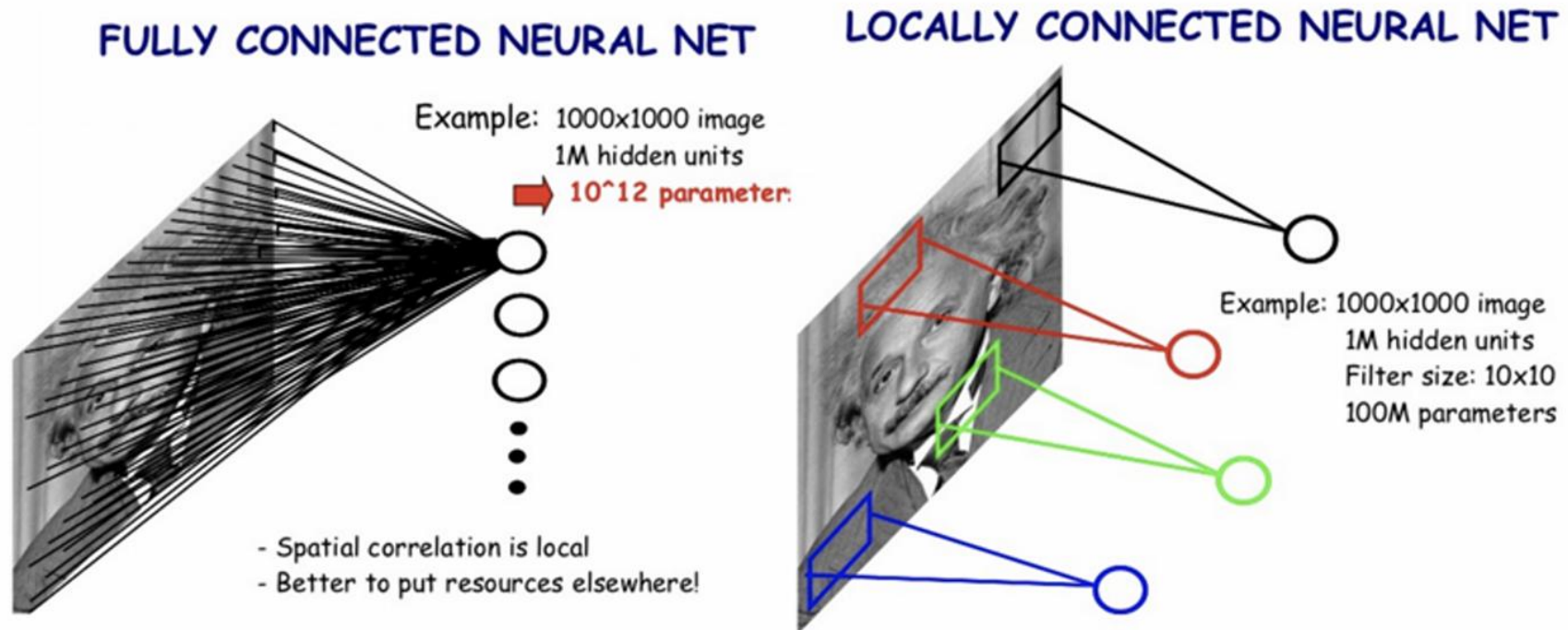
- 같은 물체지만 Crop 이후 모델이 잘못 인식 하는 경우



1.1 CNN이 등장한 배경

대규모 데이터에서 학습 어려움

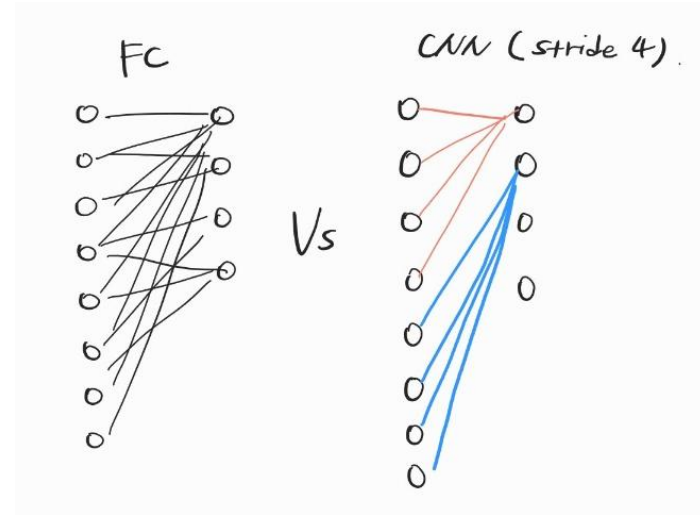
- Fully Connected Neural Network(MLP) 모델은 픽셀을 그대로 입력으로 사용했기 때문에, 고해상도 이미지에서는 연산량이 너무 많음



1.1 CNN이 등장한 배경

이를 해결하는 CNN의 특징

- Convolutional Filters를 사용해 이미지의 local 영역에서 특징을 학습
- Cnn은 신경 다발을 끊은 거다! → 효율적인 학습



- Weight Sharing을 통해 같은 필터를 이미지 전체에 적용하여 위치 불변성(spatial invariance)을 가지게 함 → 공간 정보를 유지하는 연산
- Hierarchical Feature Extraction을 통해 저수준 특징(엣지, 모서리)에서 고수준 특징(객체의 부분)까지 점진적으로 학습
- cnn도 결국 마지막에는 Feature Map을 Flatten해서 FC layer에 넣고, Conv Layer에서 이미 잘 학습된 고유한 특징들을 활용해서 최종 분류 작업을 수행함. 이 과정에서 공간적 관계는 손실되지만, 이미 Conv Layer에서 필요한 정보(특징)는 압축되었기 때문에 큰 문제가 되지 않음.
- 이러한 특징 덕분에 CNN은 단순한 template matching을 벗어나 다양한 위치, 크기, 형태의 변화를 견딜 수 있는 강력한 이미지 인식 모델로 발전

1.2 Deep dive in CNN

What is Convolution

- Convolution 개념은 CNN에서 처음 나온 것은 아니고 signal processing에서 두 개의 함수를 잘 섞어주는 operator로서 정의됨

- Continuous convolution

$$(f * g)(t) = \int f(\tau)g(t - \tau)d\tau = \int f(t - \tau)g(\tau)d\tau$$

- Discrete convolution

$$(f * g)(t) = \sum_{i=-\infty}^{\infty} f(i)g(t - i) = \sum_{i=-\infty}^{\infty} f(t - i)g(i)$$

- 2D image convolution

$$(I * K)(i, j) = \sum_m \sum_n I(m, n)K(i - m, j - n) = \sum_m \sum_n I(i - m, j - n)K(m, n)$$

- I : 전체 이미지 공간, K : 적용하고자 하는 convolution filter

1.2 Deep dive in CNN

What is Convolution

K (3x3 filter)

K_{11}	K_{12}	K_{13}
K_{21}	K_{22}	K_{23}
K_{31}	K_{32}	K_{33}

*

I (7x7 image)

I_{11}	I_{12}	I_{13}	I_{14}	I_{15}	I_{16}	I_{17}
I_{21}	I_{22}	I_{23}	I_{24}	I_{25}	I_{26}	I_{27}
I_{31}	I_{32}	I_{33}	I_{34}	I_{35}	I_{36}	I_{37}
I_{41}	I_{42}	I_{43}	I_{44}	I_{45}	I_{46}	I_{47}
I_{51}	I_{52}	I_{53}	I_{54}	I_{55}	I_{56}	I_{57}
I_{61}	I_{62}	I_{63}	I_{64}	I_{65}	I_{66}	I_{67}
I_{71}	I_{72}	I_{73}	I_{74}	I_{75}	I_{76}	I_{77}

=

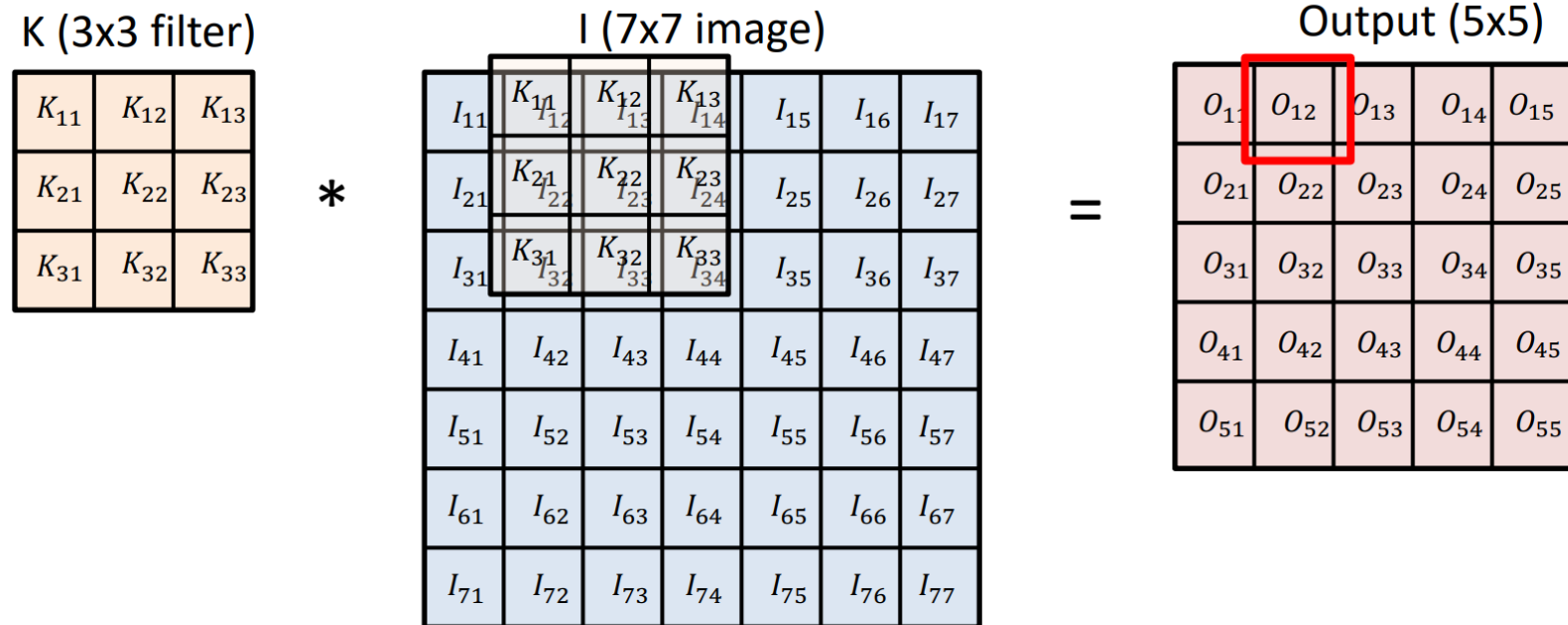
Output (5x5)

O_{11}	O_{12}	O_{13}	O_{14}	O_{15}
O_{21}	O_{22}	O_{23}	O_{24}	O_{25}
O_{31}	O_{32}	O_{33}	O_{34}	O_{35}
O_{41}	O_{42}	O_{43}	O_{44}	O_{45}
O_{51}	O_{52}	O_{53}	O_{54}	O_{55}

$$O_{11} = I_{11}K_{11} + I_{12}K_{12} + I_{13}K_{13} + I_{21}K_{21} + I_{22}K_{22} + I_{23}K_{23} + I_{31}K_{31} + I_{32}K_{32} + I_{33}K_{33} + bias$$

1.2 Deep dive in CNN

What is Convolution

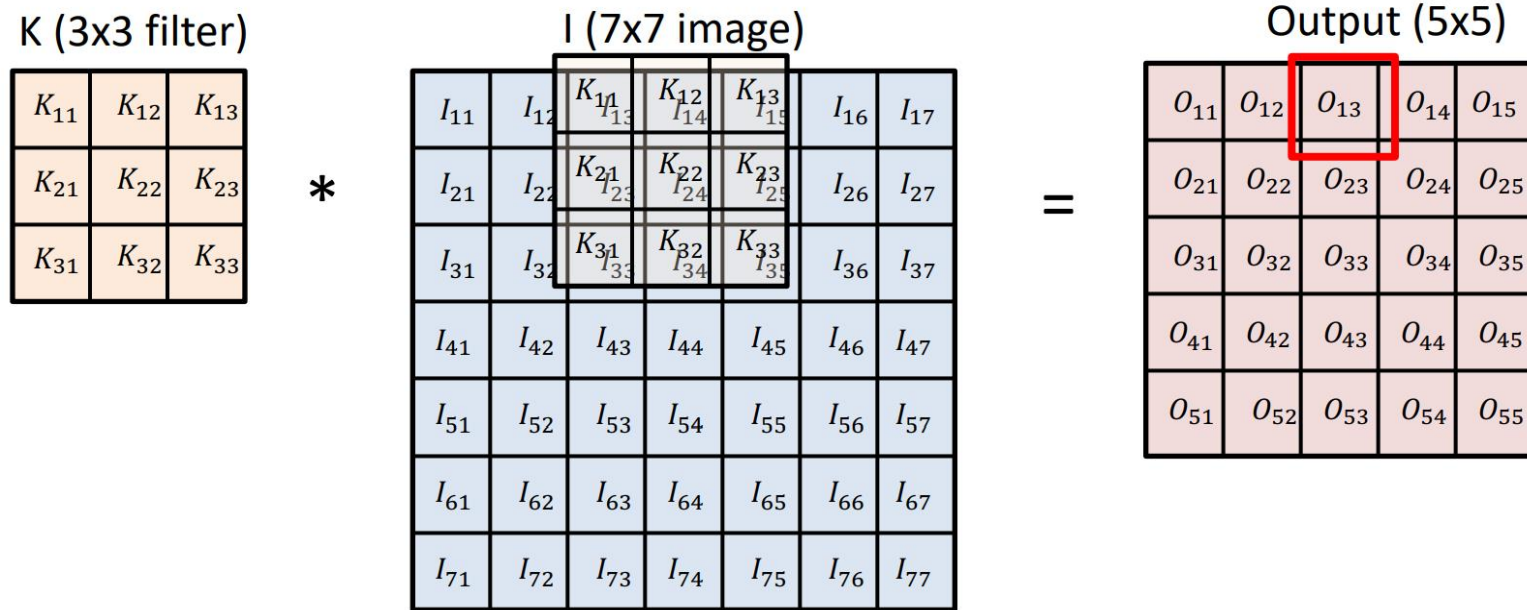


$$O_{11} = I_{11}K_{11} + I_{12}K_{12} + I_{13}K_{13} + I_{21}K_{21} + I_{22}K_{22} + I_{23}K_{23} + I_{31}K_{31} + I_{32}K_{32} + I_{33}K_{33} + bias$$

$$O_{12} = I_{12}K_{11} + I_{13}K_{12} + I_{14}K_{13} + I_{22}K_{21} + I_{23}K_{22} + I_{24}K_{23} + I_{32}K_{31} + I_{33}K_{32} + I_{34}K_{33} + bias$$

1.2 Deep dive in CNN

What is Convolution



$$O_{11} = I_{11}K_{11} + I_{12}K_{12} + I_{13}K_{13} + I_{21}K_{21} + I_{22}K_{22} + I_{23}K_{23} + I_{31}K_{31} + I_{32}K_{32} + I_{33}K_{33} + bias$$

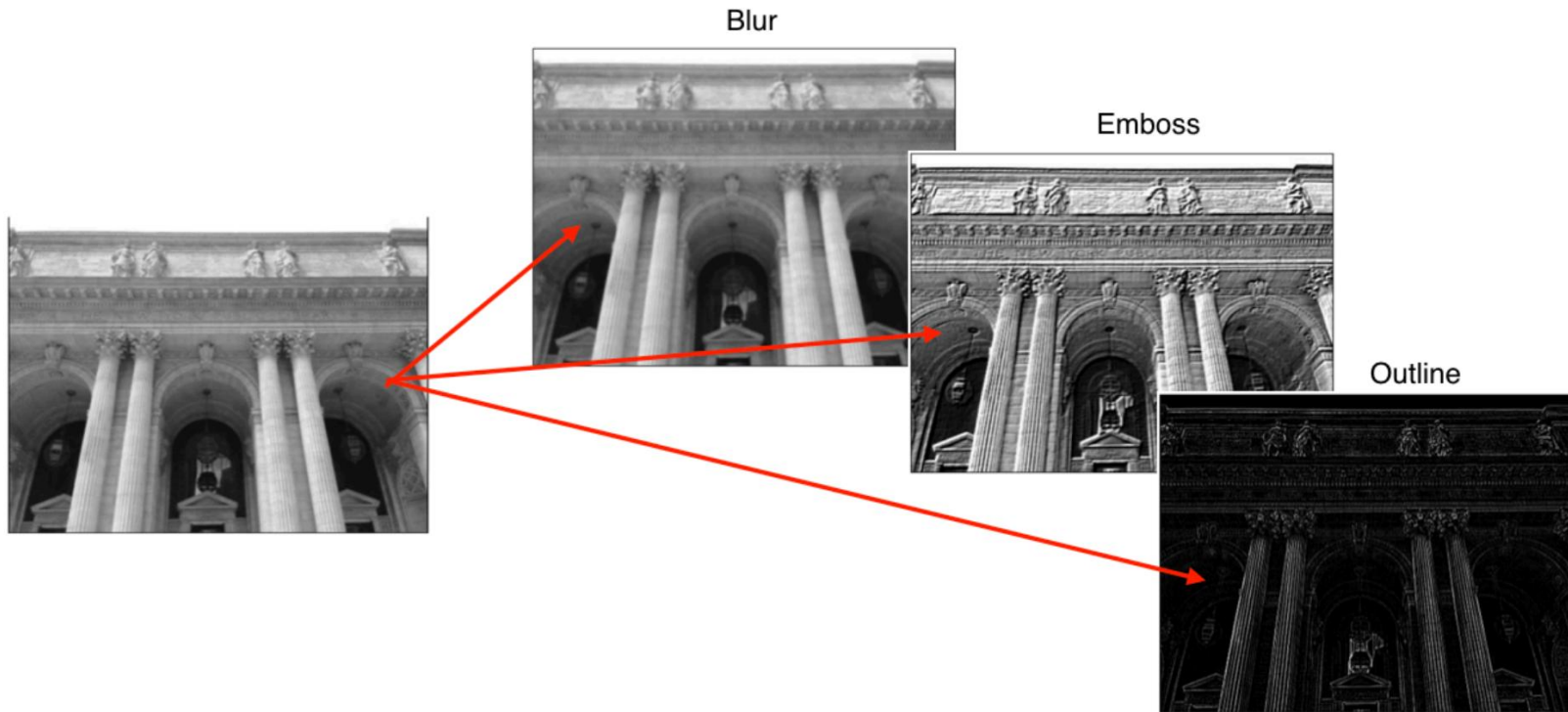
$$O_{12} = I_{12}K_{11} + I_{13}K_{12} + I_{14}K_{13} + I_{22}K_{21} + I_{23}K_{22} + I_{24}K_{23} + I_{32}K_{31} + I_{33}K_{32} + I_{34}K_{33} + bias$$

$$O_{13} = I_{13}K_{11} + I_{14}K_{12} + I_{15}K_{13} + I_{23}K_{21} + I_{24}K_{22} + I_{25}K_{23} + I_{33}K_{31} + I_{34}K_{32} + I_{35}K_{33} + bias$$

1.2 Deep dive in CNN

2D convolution in action

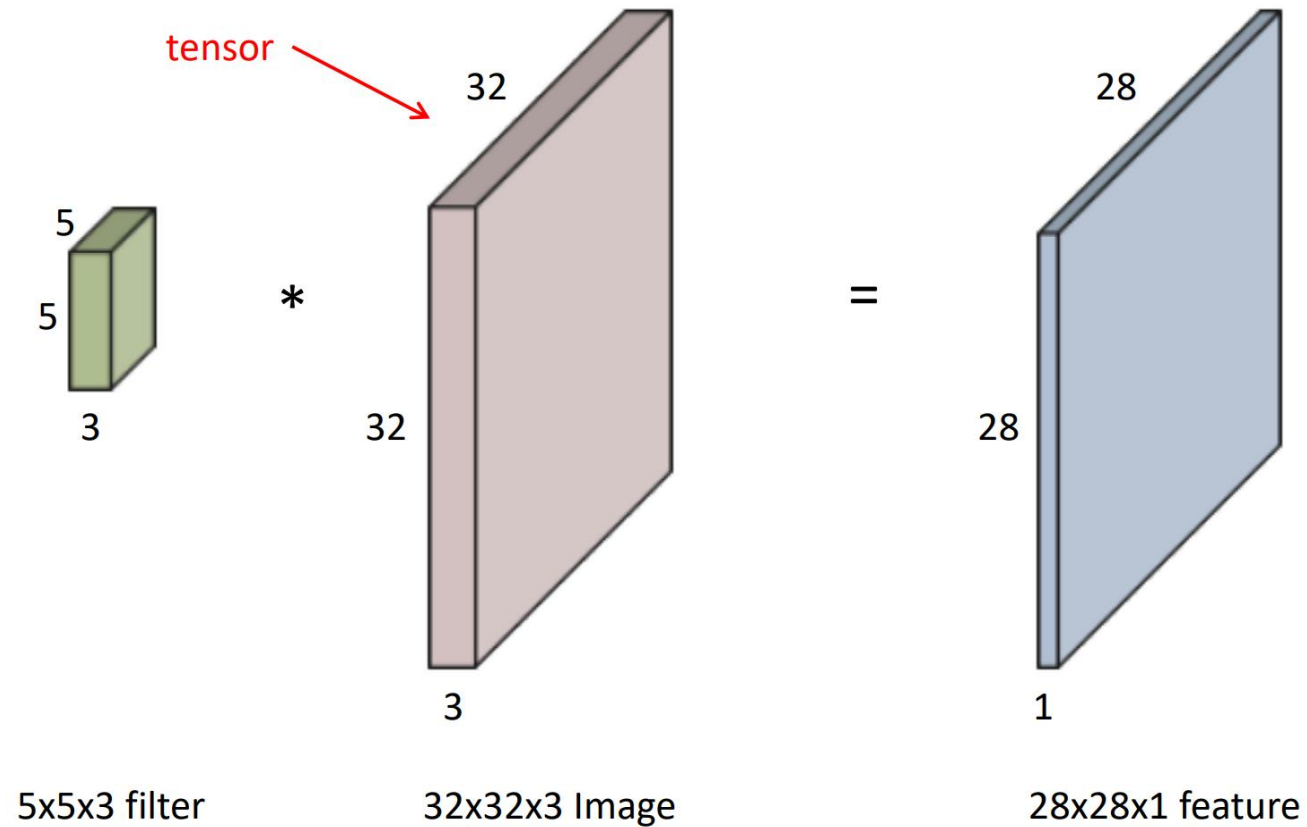
- 적용 filter에 따라 convolution output이 달라짐
- Ex) 3x3 filter, filter의 모든 값이 1/9 $\rightarrow$ 3x3 이미지 픽셀 값의 평균이 convolution filter 통과한 output $\rightarrow$ 이미지가 특정 영역 만큼의 픽셀을 합쳐서 평균 $\rightarrow$ blur 효과



1.2 Deep dive in CNN

RGB Image Convolution

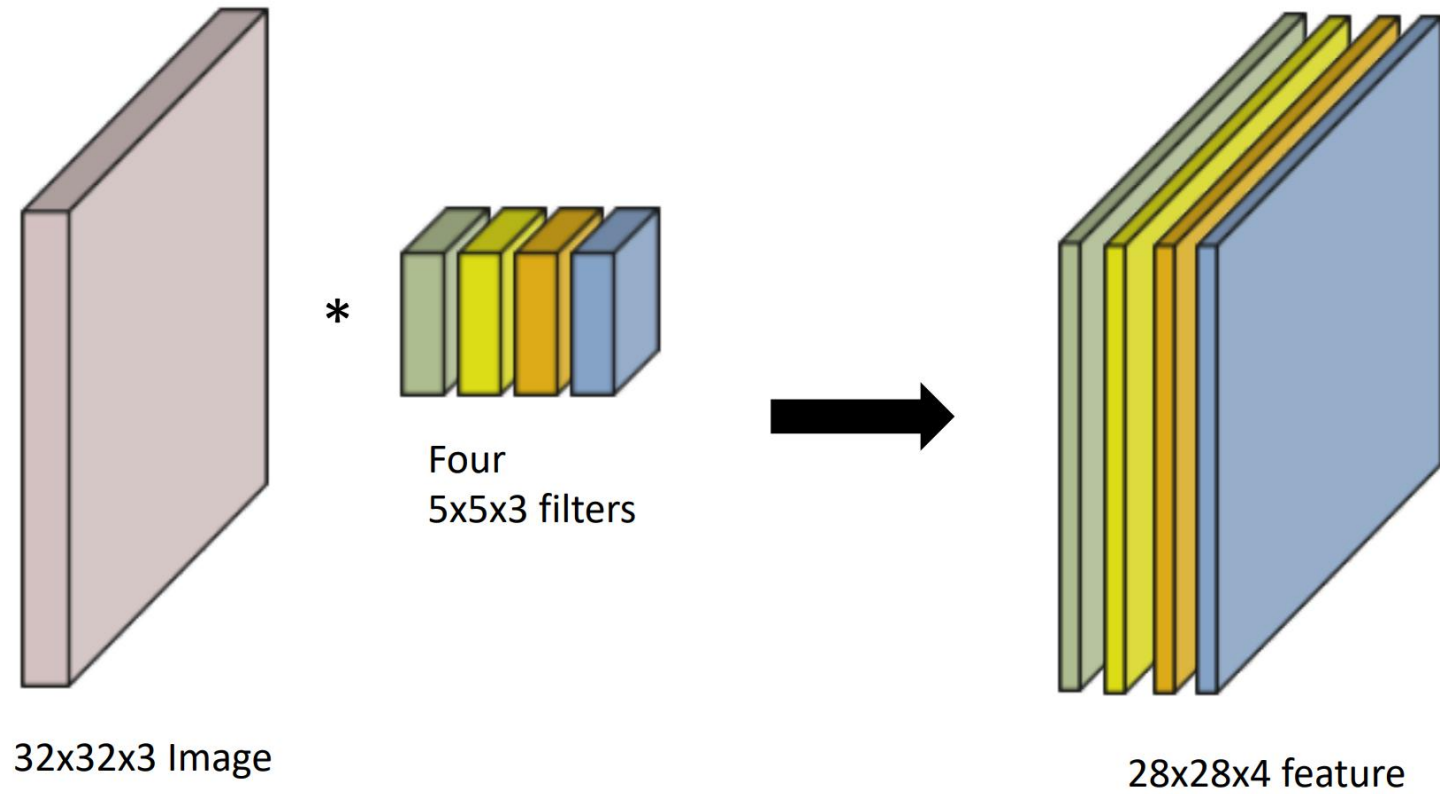
- Filter의 채널 수(depth)는 Image의 채널 수를 따라감



1.2 Deep dive in CNN

RGB Image Convolution

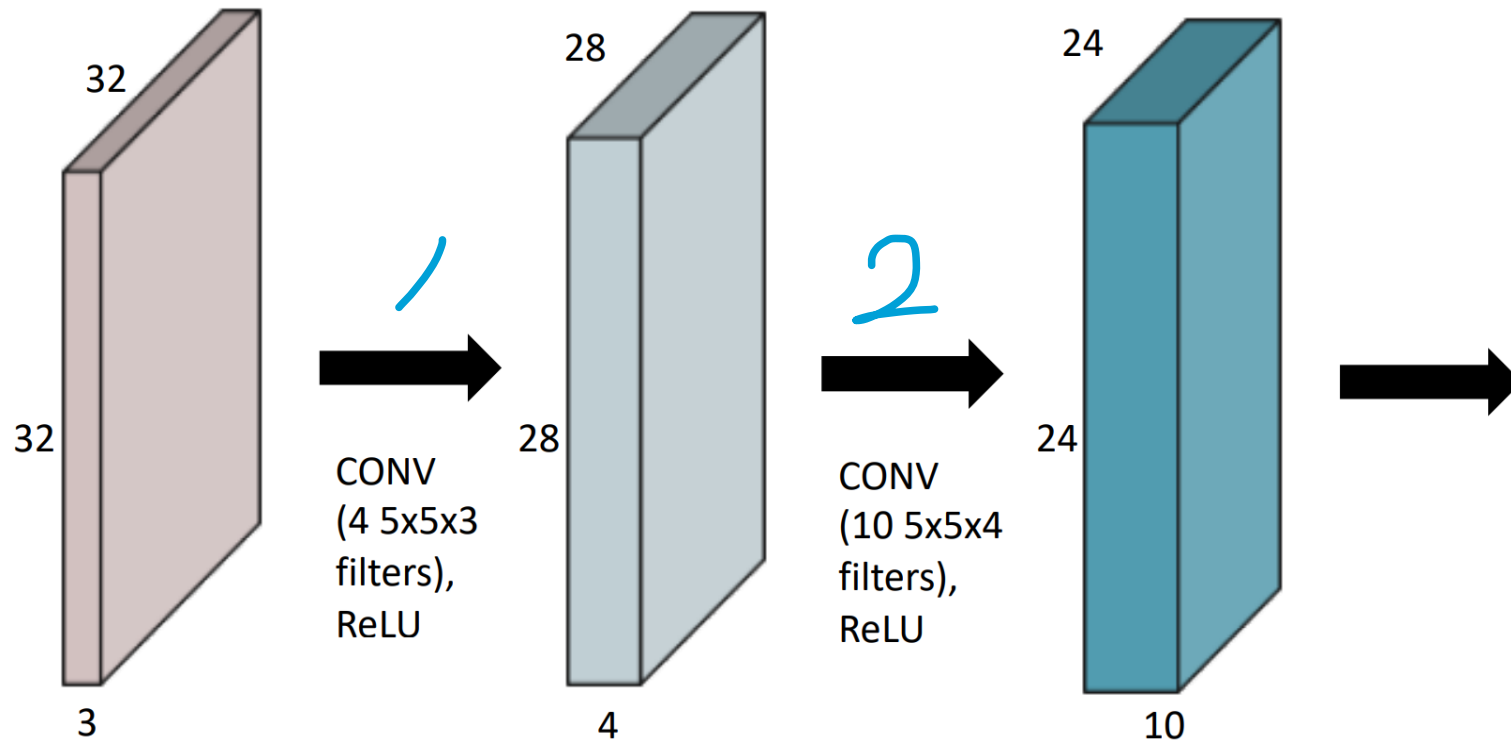
- filter 개수 = Output feature map 채널 수



1.2 Deep dive in CNN

Stack of Convolutions

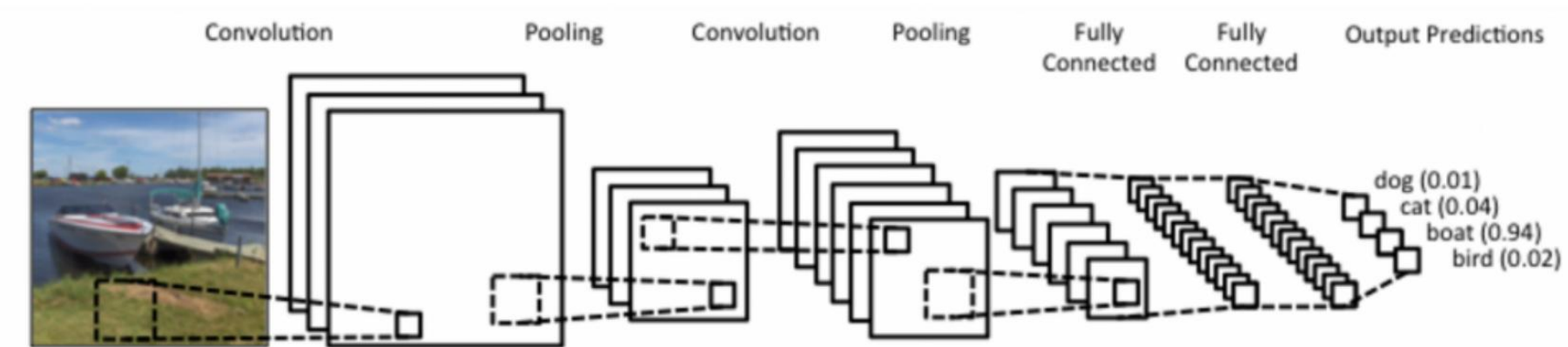
- convolution 이후 나오는 feature map의 각 element 별로 (nonlinear) activation 통과
- 과정 1 연산에 필요한 파라미터 수 : $5 \times 5 \times 3 \times 4$
- 과정 2 연산에 필요한 파라미터 수 : $5 \times 5 \times 4 \times 10$



1.2 Deep dive in CNN

Convolutional Neural Networks

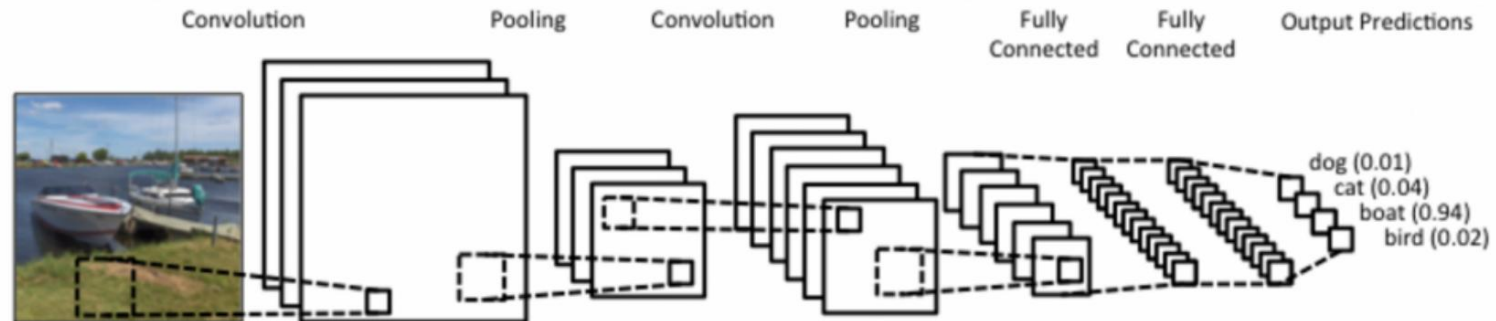
- 일반적인(고전적인) CNN 구성 요소 : convolution layer, pooling layer, fully connected layer
- convolution and pooling layers : feature extraction
- Fully connected layer : decision making (e.g., classification)
- 최근 들어 뒷단의 FC layer가 점점 최소화되는 추세 → 파라미터 수에 굉장히 dependent한 layer라서
- 학습하고자 하는 모델의 파라미터 수가 늘어날수록 학습이 어렵고 generalization performance가 떨어짐 → CNN 발전 방향 : 최대한 모델을 Deep 하게(conv layer를 최대한 많이) 하지만 동시에 파라미터 수를 줄이는 데 집중 → 여러 technique



1.2 Deep dive in CNN

Convolutional Neural Networks - 학습 방법

- cnn에서는 filter가 뉴런의 weight임. 즉, filter의 값을 ai가 자동 학습하도록 함
- Forward Propagation
 - 입력 이미지가 필터를 거쳐 특징 맵(feature map) 이 생성됨.
 - 활성화 함수(ReLU 등)를 거친 후, 풀링(pooling) 등을 적용하여 중요한 특징을 추출.
 - 마지막에는 Fully Connected Layer(FC layer)를 거쳐 최종 출력을 얻음.
- 모델의 예측값과 실제 정답(레이블) 간의 차이를 계산하여 손실(loss) 을 구함.
- Backpropagation, Gradient Descent로 Weight Update
 - 필터 값(가중치)이 점진적으로 조정되면서, 이미지의 중요한 특징을 잘 잡아내도록 학습됨.



1.2 Deep dive in CNN

Convolution Arithmetic (of GoogLeNet)

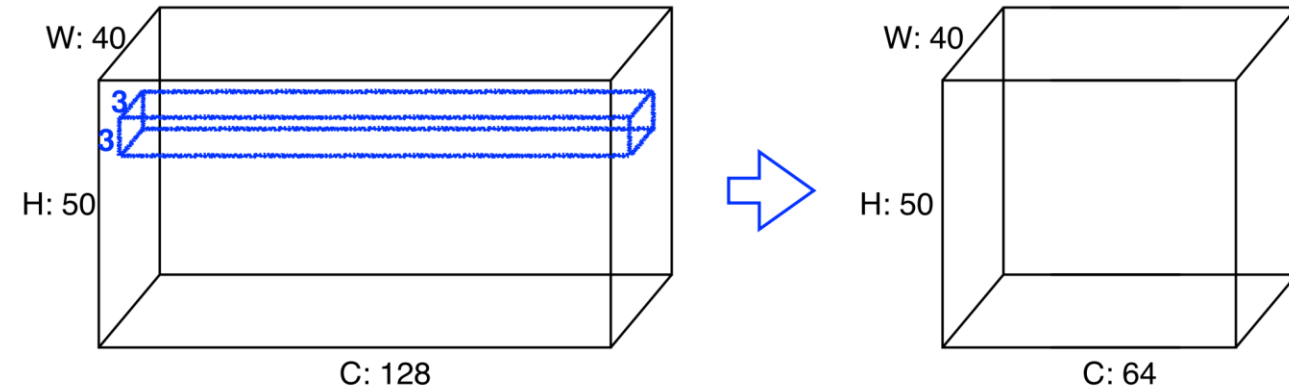
- NN architecture를 볼 때 layer별로 몇 개의 파라미터로 이루어져 있고 전체 파라미터 수가 몇 개 인지에 대한 감을 가지고 있는 것이 중요

Stride		Channel								#param	
type	patch size/ stride	output size	depth	#1×1	#3×3 reduce	#3×3	#5×5 reduce	#5×5	pool proj	params	ops
convolution	7×7/2	112×112×64	1							2.7K	34M
max pool	3×3/2	56×56×64	0								
convolution	3×3/1	56×56×192	2		64	192				112K	360M
max pool	3×3/2	28×28×192	0								
inception (3a)		28×28×256	2	64	96	128	16	32	32	159K	128M
inception (3b)		28×28×480	2	128	128	192	32	96	64	380K	304M
max pool	3×3/2	14×14×480	0								
inception (4a)		14×14×512	2	192	96	208	16	48	64	364K	73M
inception (4b)		14×14×512	2	160	112	224	24	64	64	437K	88M
inception (4c)		14×14×512	2	128	128	256	24	64	64	463K	100M
inception (4d)		14×14×528	2	112	144	288	32	64	64	580K	119M
inception (4e)		14×14×832	2	256	160	320	32	128	128	840K	170M
max pool	3×3/2	7×7×832	0								
inception (5a)		7×7×832	2	256	160	320	32	128	128	1072K	54M
inception (5b)		7×7×1024	2	384	192	384	48	128	128	1388K	71M
avg pool	7×7/1	1×1×1024	0								
dropout (40%)		1×1×1024	0								
linear		1×1×1000	1							1000K	1M
softmax		1×1×1000	0								

1.2 Deep dive in CNN

Convolution Arithmetic

- Padding (1), Stride (1), 3x3 Kernel
- 네트워크 구조만 보고 파라미터 개수(만 단위, 십 만, 백 만 단위..) 를 감 잡는 것이 중요
- 모델 크기를 대략적으로 알면 훈련 시간과 필요 자원을 빠르게 예측할 수 있음
 - 몇만 개(10^4) 파라미터 → 빠르게 학습, 작은 데이터셋에서도 가능
 - 백만 개(10^6) 파라미터 → 일반적인 CNN 모델 수준, GPU 필요 가능성 높음
 - 수천만 개($10^7 \sim 10^8$) 이상 → 대형 모델, 고성능 GPU 필요
- 적절한 모델 선택 및 과적합(Overfitting) 가능성 판단
 - 너무 많은 파라미터를 가진 모델을 작은 데이터셋에서 학습하면 과적합 위험이 높아짐
 - 반대로, 파라미터가 적으면 복잡한 패턴을 학습하지 못해 언더피팅 위험이 있음
 - 작은 데이터셋 → 10만 개 이하(10^5) 파라미터 모델 적절
 - 큰 데이터셋 → 수천만 개($10^7 \sim 10^8$) 파라미터 모델 고려 가능



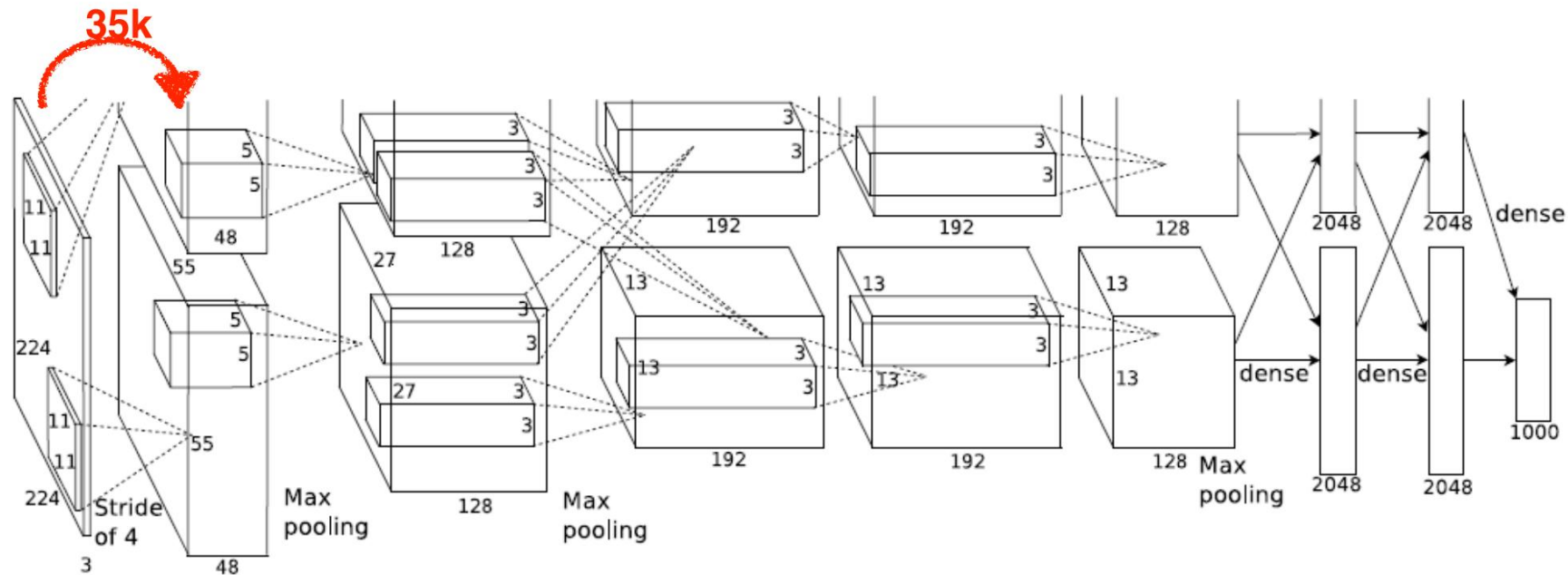
What is the **number of parameters** of this model?

The answer is $3 \times 3 \times 128 \times 64 = 73,728$

1.2 Deep dive in CNN

Convolution Arithmetic example(AlexNet)

- 2가 곱해지는 이유는 사실 96 채널의 feature map을 만들어야 하는데 gpu 메모리가 크지 않아서 하나의 gpu 메모리 사이즈에 맞추다 보니 48채널을 2개 만든 것
- 1번째 layer의 파라미터 : 약 35k



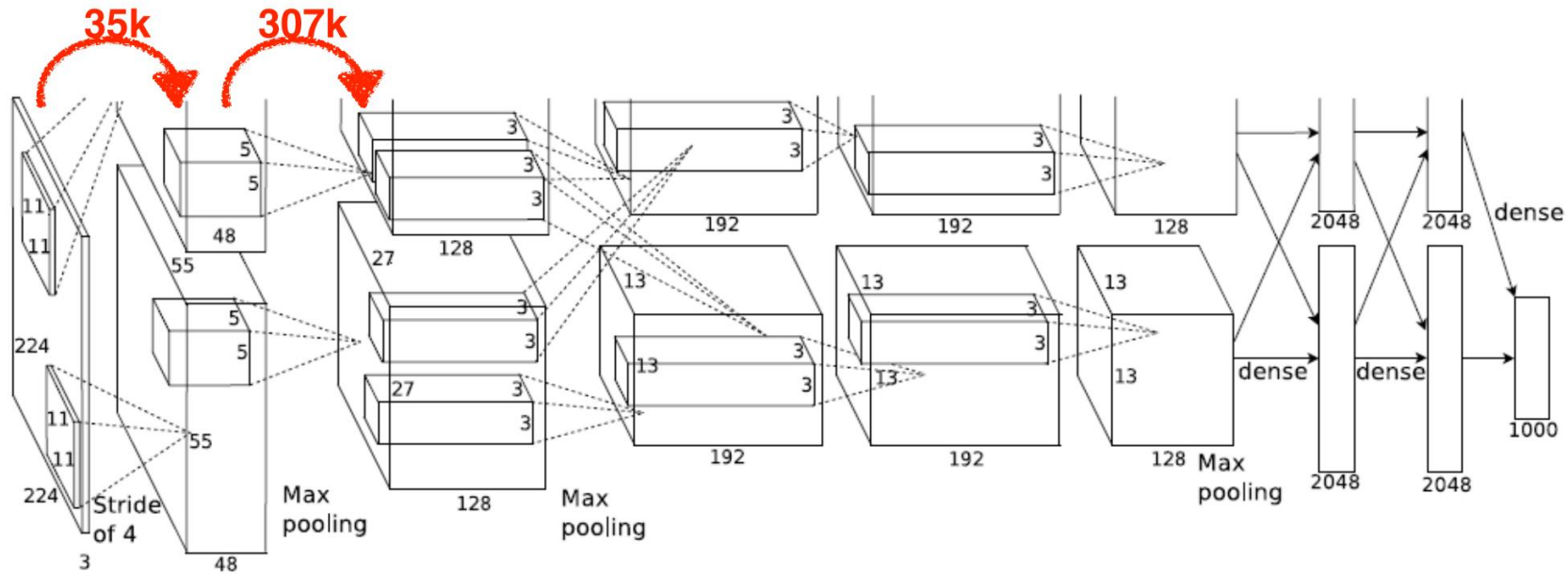
$$11 \times 11 \times 3 \times 48 \times 2 \approx 35k$$

What is the **number of parameters** of this model?

1.2 Deep dive in CNN

Convolution Arithmetic example(AlexNet)

- 2번째 layer의 파라미터 : 약 307k



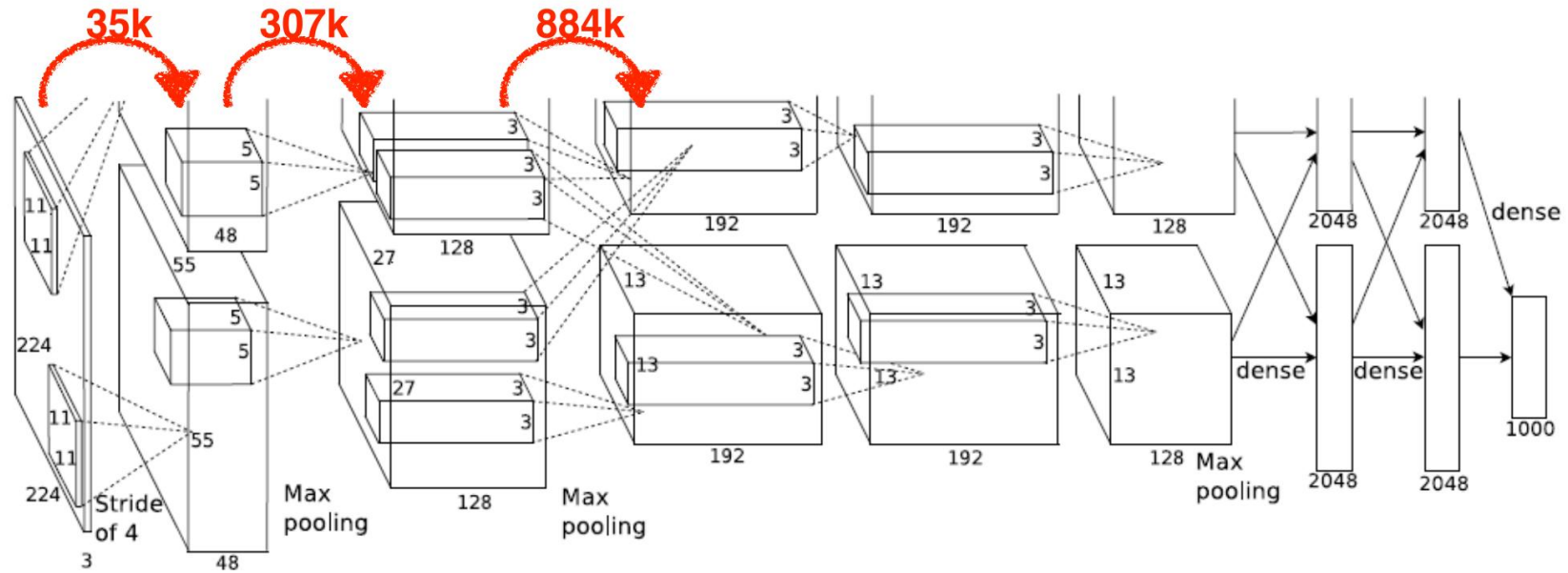
$$5 \times 5 \times 48 \times 128 * 2 \approx 307k$$

What is the **number of parameters** of this model?

1.2 Deep dive in CNN

Convolution Arithmetic example(AlexNet)

- 3번째 layer의 파라미터 : 약 884k
- 위, 아래 네트워크가 interchange 되기 때문에 *2, *2 가 추가됨



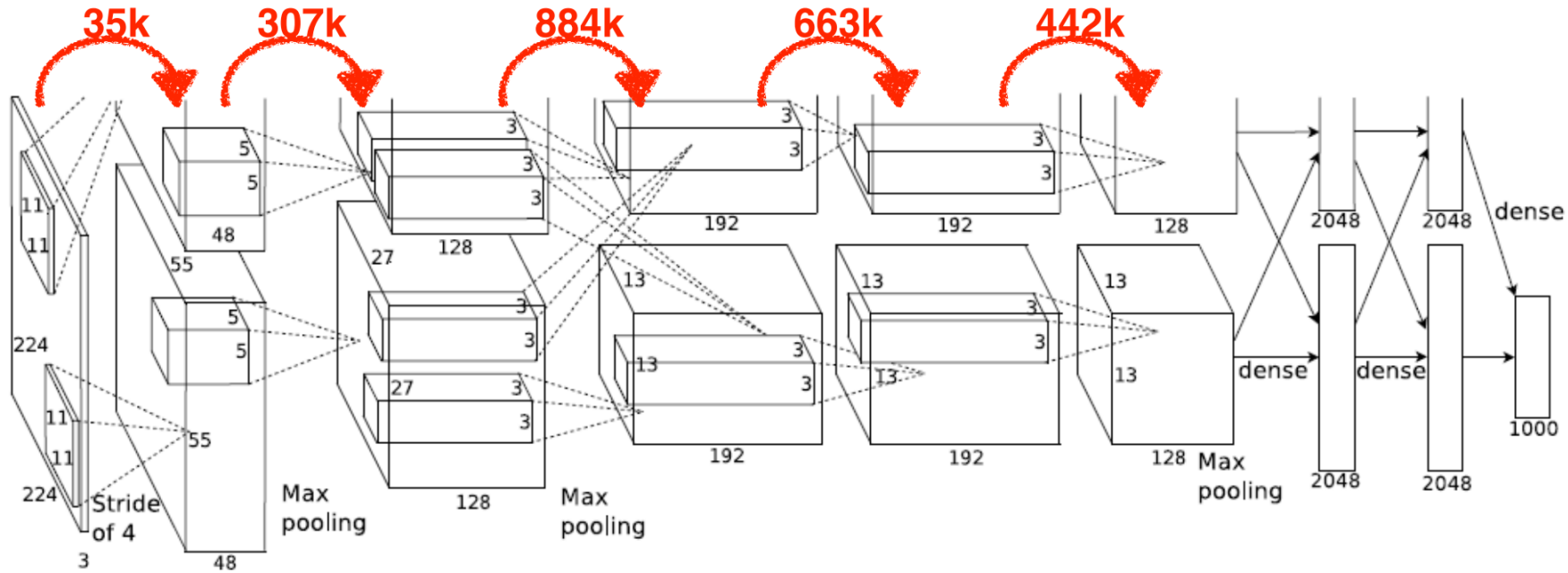
$$3 \times 3 \times 128 \times 2 \times 192 \times 2 \approx 884k$$

What is the **number of parameters** of this model?

1.2 Deep dive in CNN

Convolution Arithmetic example(AlexNet)

- 5번째 layer의 파라미터 : 약 442k

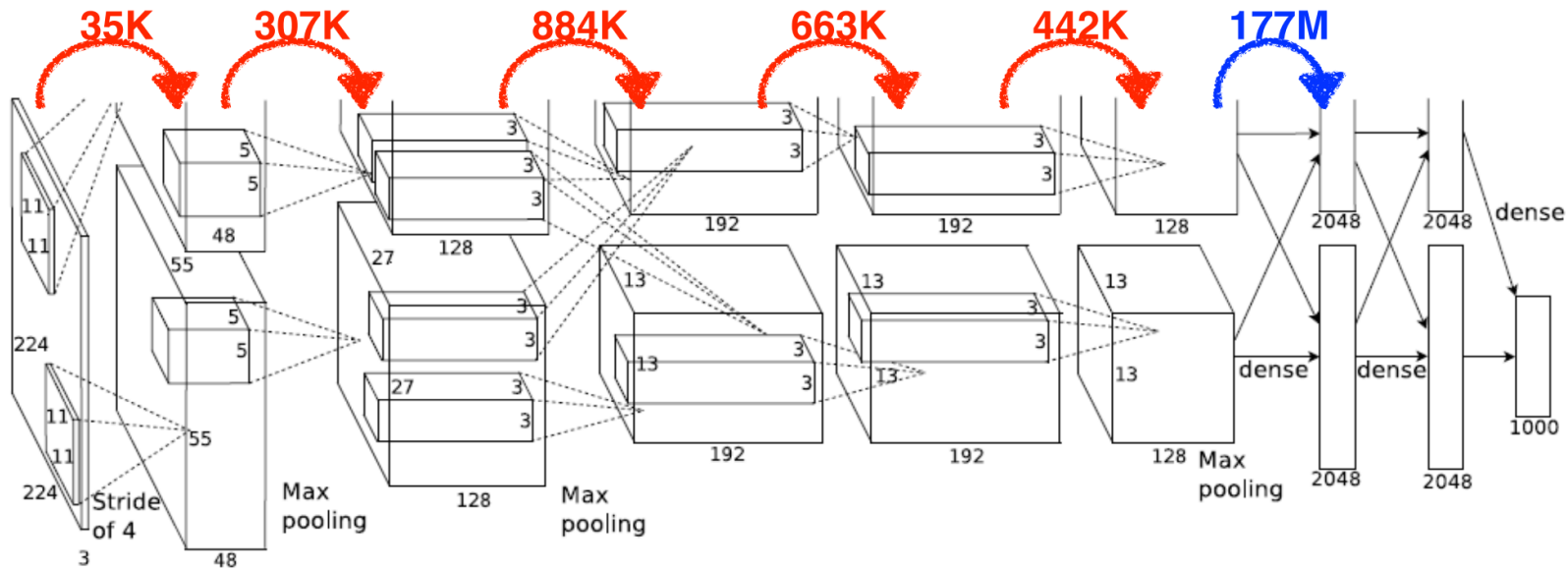


$$3 \times 3 \times 192 \times 128 * 2 \approx 442k$$

1.2 Deep dive in CNN

Convolution Arithmetic example(AlexNet)

- 6번째 layer의 파라미터 (Dense layer = FC layer) : 약 177M
- 위, 아래 네트워크 interchange 됨



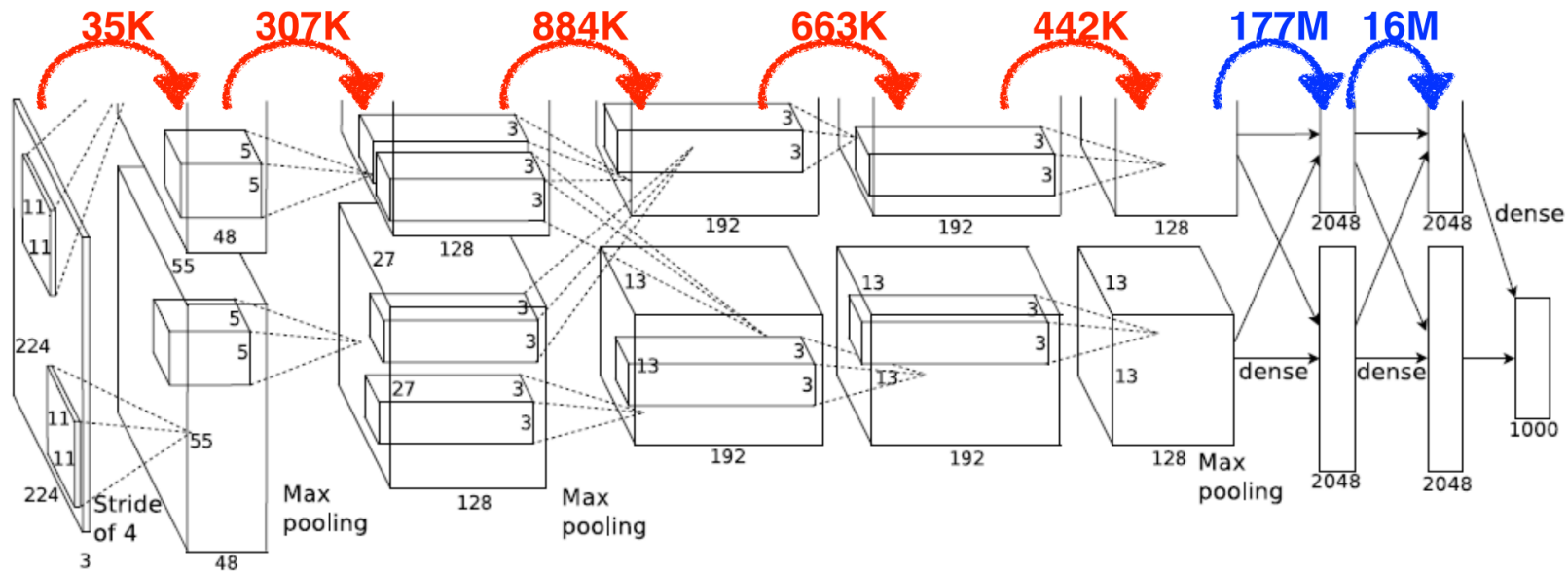
$$13 * 13 * 128 * 2 \times 2048 * 2 \approx 177M$$

What is the **number of parameters** of this model?

1.2 Deep dive in CNN

Convolution Arithmetic example(AlexNet)

- 7번째 layer의 파라미터 : 약 16M



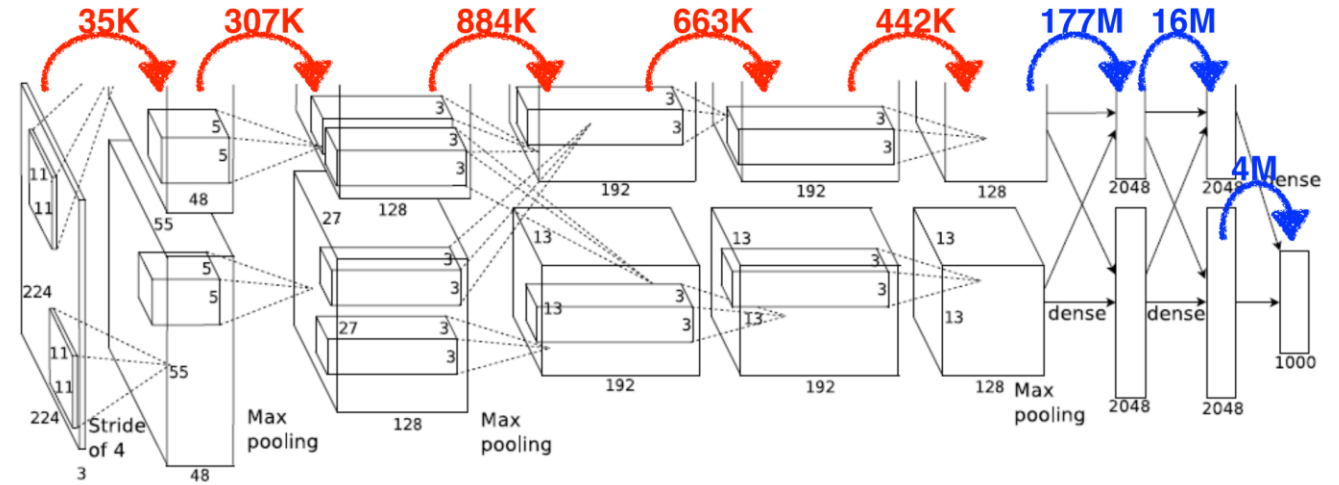
$$2048 * 2 \times 2048 * 2 \approx 16M$$

What is the **number of parameters** of this model?

1.2 Deep dive in CNN

Convolution Arithmetic example(AlexNet)

- 8번째 layer의 파라미터 : 약 4M
- 빨간색 layer : convolution layer
- 파란색 layer : dense layer(FC layer)
- 파라미터 수가 conv layer에 비해 dense layer가 약 1000배
- Dense layer가 일반적으로 훨씬 더 많은 파라미터를 갖는 이유 :
 - Convolution 연산에서 각 kernel(filter)가 모든 위치에서 동일하게 적용되기 때문
 - kernel은 shared parameter(weight, bias) :
같은 kernel이 이미지 오른쪽 위, 왼쪽 아래 등 모두 동일하게 적용



$$2048 * 2 * 1000 \approx 4M$$

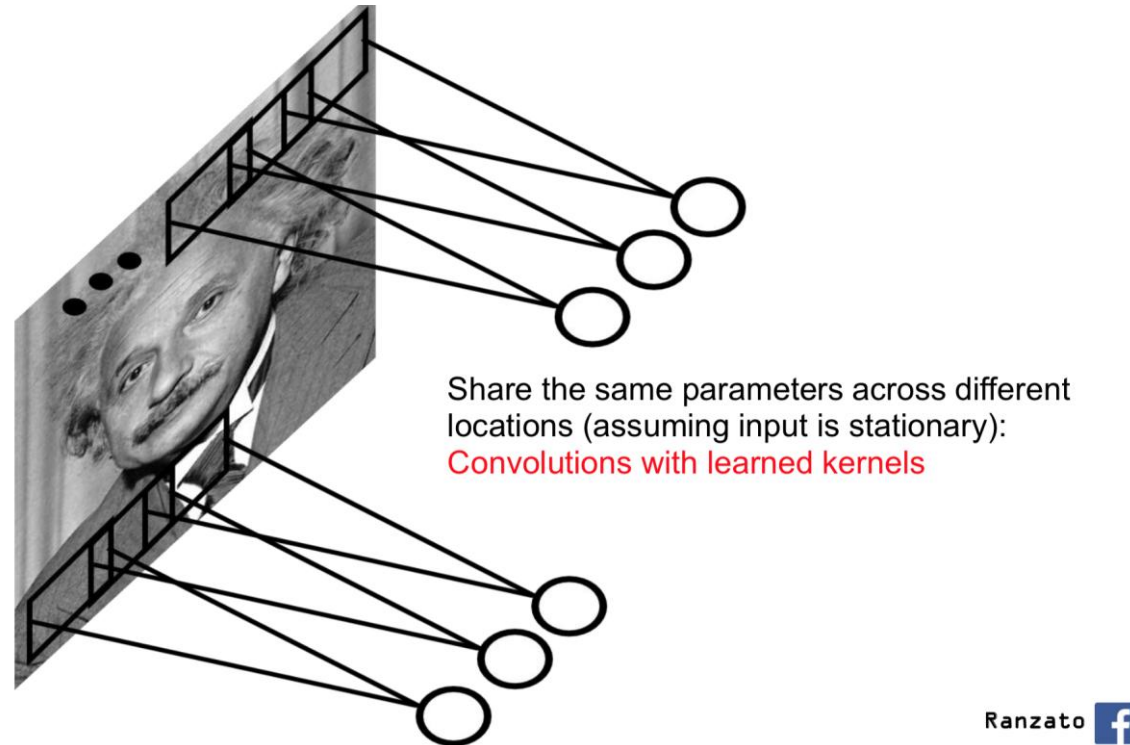
What is the **number of parameters** of this model?

- NN의 성능을 올리기 위해 파라미터를 줄이는 것이 중요한데 대부분의 파라미터가 FC layer에 들어있기 때문에 NN의 발전 방향이 뒷단의 FC layer를 최대한 줄이고 앞의 conv layer를 깊게 쌓는 방식이 됨
- 여러 technique을 통해 NN의 layer는 깊어지지만 파라미터 수는 점점 줄어들게 됨 (성능은 up)

1.2 Deep dive in CNN

CNN의 특징 - Weight Sharing

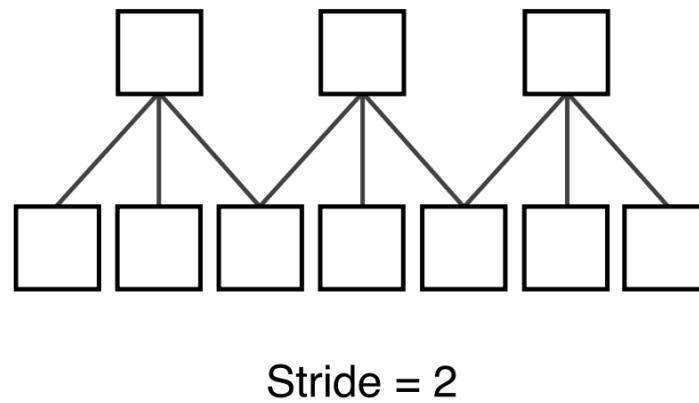
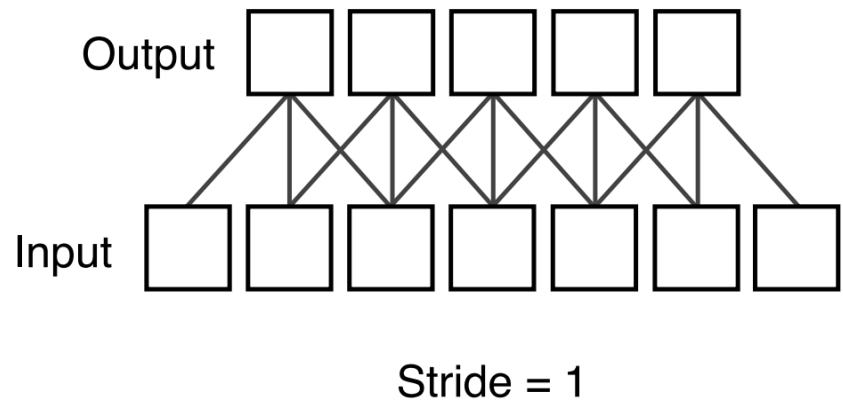
- CNN에서는 동일한 필터(가중치 집합)를 이미지의 모든 영역에 적용하는데, 이는 하나의 필터가 이미지의 모든 부분에서 동일한 역할을 한다는 의미
- 이 방식은 필터가 이미지 내의 위치와 상관없이 동일한 특징(ex. 사람 귀)을 인식하도록 도와주며, 객체가 이미지 내 어디에 있든 동일한 패턴을 추출할 수 있게 함



1.2 Deep dive in CNN

CNN의 특징 - 1d Stride (큰 걸음, 성큼성큼 걷다)

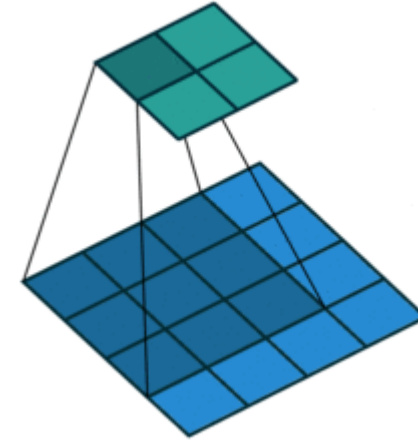
- Stride=1 : convolution filter(kernel)을 한 픽셀씩 옮겨서 연산
- 1d stride
- Size 3 kernel



1.2 Deep dive in CNN

CNN의 특징 - 2d Stride (큰 걸음, 성큼성큼 걷다)

- 2d stride에서는 stride의 파라미터가 2개(width, height 방향)
- $\text{stride}=(sh, sw)$
- 보통은 $\text{stride}=(s, s)$ 처럼 너비(w), 높이(h)에 동일한 값 사용
- 특정 상황에서는 다르게 설정할 수도 있음
- Ex) 텍스트 인식 모델에서 글자가 주로 가로 방향으로 배열될 경우
 - (1, 2) stride : 높이 방향(stride_height)으로는 1씩, 너비 방향(stride_width)으로는 2씩 이동
 - 가로 방향으로 두 칸씩 이동하면서 특징을 추출하므로, 한 번에 더 넓은 단어 패턴을 감지할 수 있음
 - 가로로 한번에 더 많은 영역을 커버하면서, 세로 정보 손실을 최소화



1.2 Deep dive in CNN

Stride의 의미

1. 정보를 요약하는 과정 (Summarization)

- 작은 stride(예: 1)는 이미지의 세부 정보를 최대한 보존하려는 접근. 모든 픽셀을 면밀히 살펴보고, 미세한 특징을 유지하는 것을 목표
- 큰 stride(예: 2 이상)는 정보를 더 압축하고 요약하는 과정. 특정 부분을 생략하면서도 전체적인 패턴을 학습하려는 시도

2. 추상화와 해상도의 절충 (Abstraction vs. Resolution)

- 작은 stride를 사용하면 높은 해상도로 정보를 유지할 수 있지만, 연산량이 많아지고 속도가 느림
- 큰 stride는 해상도를 낮추고 연산량을 줄이는 대신, 더 추상적인 특징을 학습하도록 유도
- 인간이 사물을 인식할 때 세부적인 디테일을 볼지, 전체적인 형상을 파악할지를 선택하는 과정과 비슷

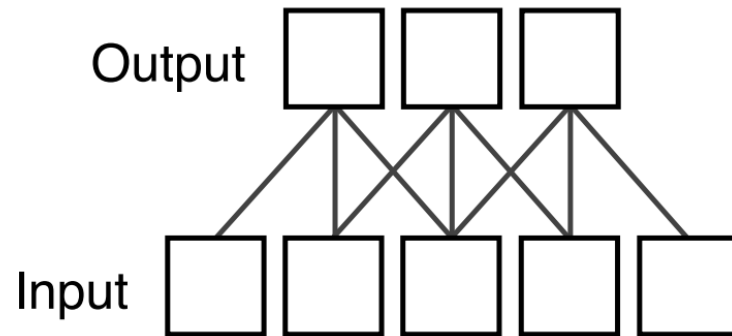
3. 정리

- 필터(커널)가 한 번의 이동에서 얼마나 많은 픽셀을 건너뛰는지를 결정하는 하이퍼파라미터
- 세밀한 특징과 전체적인 패턴을 어떻게 균형 있게 학습할 것인가를 결정하는 요소

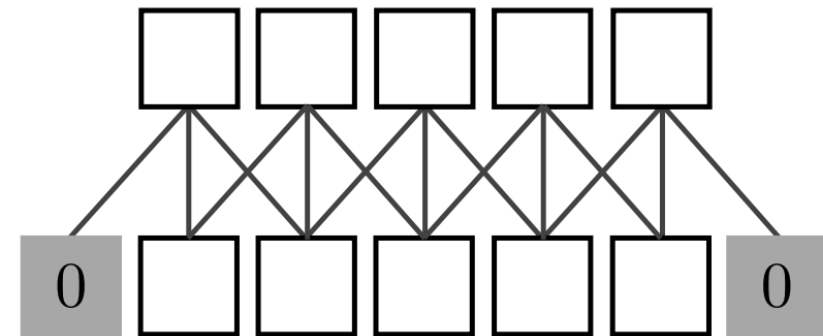
1.2 Deep dive in CNN

CNN의 특징 - Padding

- Input image에 conv 연산을 하면 output이 input 사이즈랑 똑같이 안 나옴 → **boundary** 정보가 버려지기 때문에
- 가장자리에도 conv 연산을 하려면 padding으로 값을 채워야 함



No padding (stride=1)



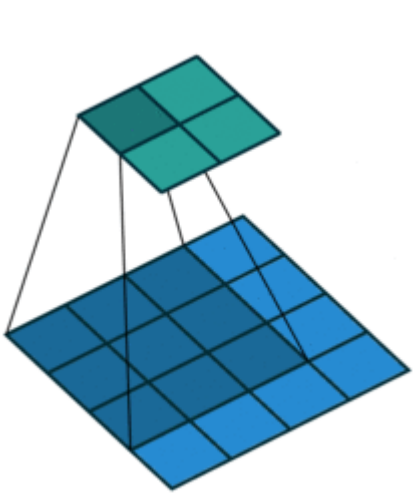
Zero padding (stride=1)

- Input과 output의 spatial dim이 똑같아 짐

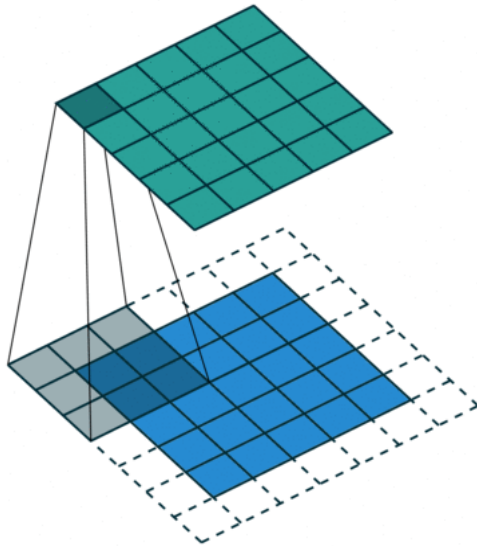
1.2 Deep dive in CNN

CNN의 특징 - Stride & Padding

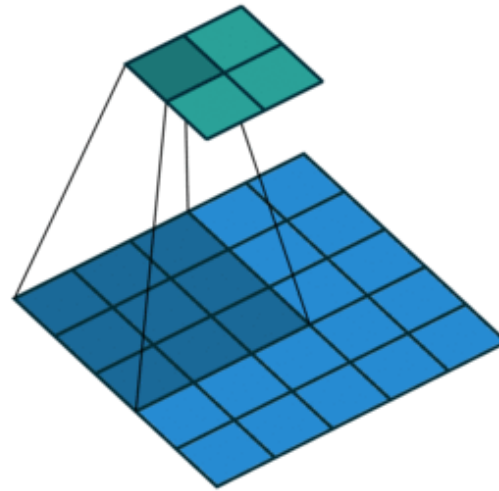
- 적절한 크기의 padding, stride를 설정하면 input과 feature map의 spatial dim이 같아짐
- Ex1) padding 1, stride 1, 3x3 conv
- Ex2) padding 2, stride 1, 5x5 conv
- 원하는 출력 size에 맞춰서 padding과 stride를 설정 가능



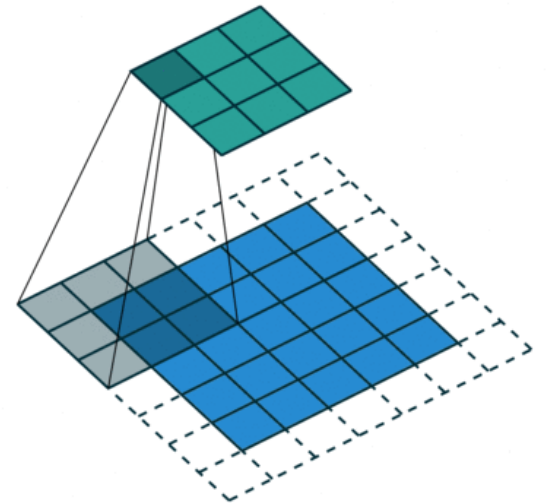
Stride (1)
Padding (0)



Stride (1)
Padding (1)



Stride (2)
Padding (0)



Stride (2)
Padding (1)

1.2 Deep dive in CNN

■ 왜 Input과 Feature Map의 Spatial Dimension을 동일하게 유지?

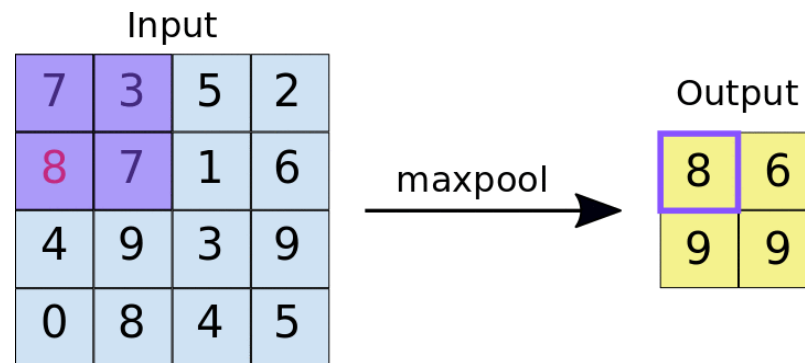
- 네트워크가 깊어지더라도 공간적인 정보가 일관되게 전달
- 위치적 일관성(Position Consistency)을 유지
 - 특정 영역의 픽셀들이 여러 계층을 거쳐도 자신의 상대적 위치를 잃지 않고 유지되기 때문에 위치 정보가 왜곡되지 않음
- 깊은 네트워크에서도 안정적인 학습 가능
 - 네트워크가 깊어지더라도 특징 맵의 크기가 급격히 줄어들지 않으면서도 정보 손실이 최소화

1.2 Deep dive in CNN

CNN의 특징 - Pooling (어떤 정보를 버리고 어떤 정보를 유지할 것인가)

- 공간적 축소 & 불변성
 - Max Pooling과 같은 풀링 층은 입력된 이미지의 일부 영역을 대표하는 값을 추출하여 특징 맵의 크기를 축소
 - 이 과정에서 중요한 특징을 유지하면서 위치 불변성을 더 강화할 수 있음
 - 풀링을 통해 이미지는 일부 지역 정보만으로도 객체의 특징을 충분히 인식할 수 있기 때문에, 객체의 위치가 변경되더라도 잘 인식할 수 있게 됨
- 정보 요약 & 일반화
 - 세부적인 정보 대신 핵심 패턴을 학습하게 하여 모델의 일반화 성능을 향상
- 계층적 표현 학습
 - 낮은 수준의 특징(모서리, 질감 등)을 점진적으로 더 높은 수준의 개념(객체, 의미적 패턴 등)으로 연결하는 과정

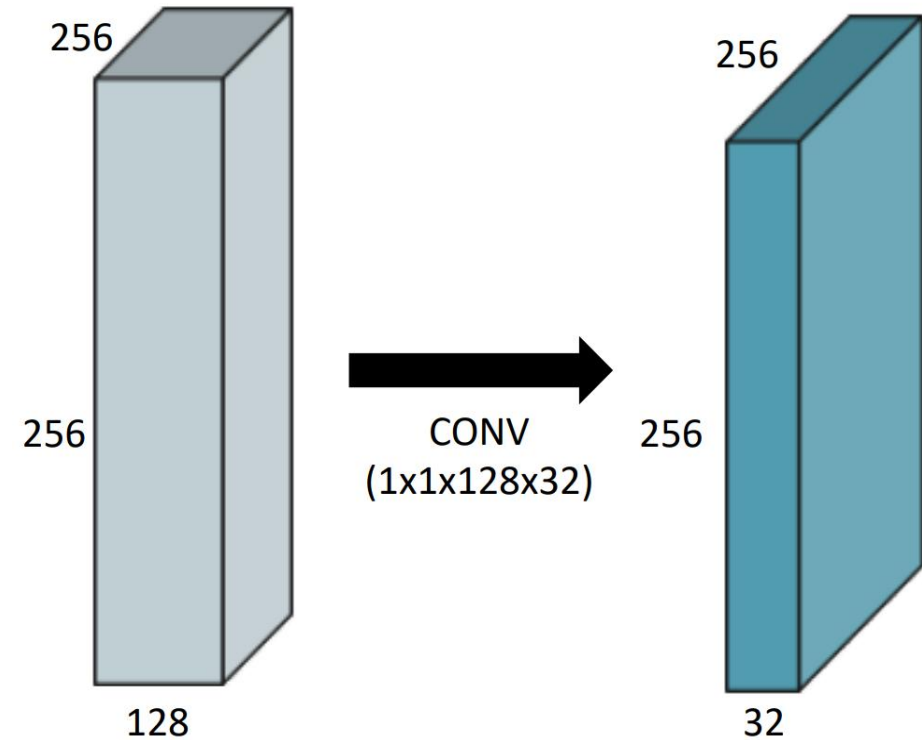
- global avg pool, avg pool, max pool..



1.2 Deep dive in CNN

CNN의 특징 - 1x1 Convolution

- 1x1 conv은 이미지의 어떤 영역을 보는 것은 아님
- 1x1 의미 자체가 한 픽셀만 본다는 것
- 채널 방향으로 줄이거나 늘리거나 할 때 사용
- Ex) Dimension reduction
- 256 x 256의 spatial dim은 유지하고 채널만 128에서 32로 줄임
- To reduce the number of parameters while increasing the depth(layer)
- e.g., bottleneck architecture
 - 1x1 conv를 이용해서 NN을 깊게 쌓는데 파라미터 수를 줄이는 방법



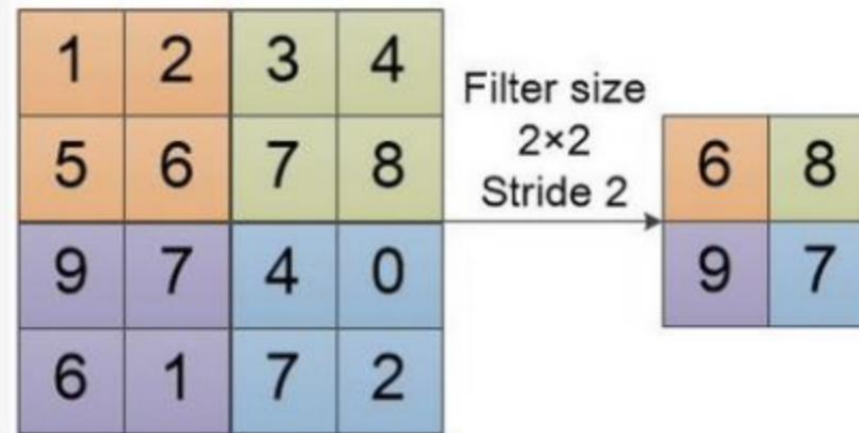
- 학습하고자 하는 모델의 파라미터 수가 늘어날수록 학습이 어렵고 generalization performance가 떨어짐 → CNN 발전 방향 : 최대한 모델을 Deep 하게(conv layer를 최대한 많이) 하지만 동시에 파라미터 수를 줄이는 데 집중 → 여러 technique

1.2 Deep dive in CNN

CNN의 특징 - conv랑 pooling 이후 output feature map size 계산 공식

- I : 입력 feature map 크기
- F : filter 크기(kernel size)
- P : Padding
- S : Stride
- Input의 가로, 세로 길이는 동일하다고 가정

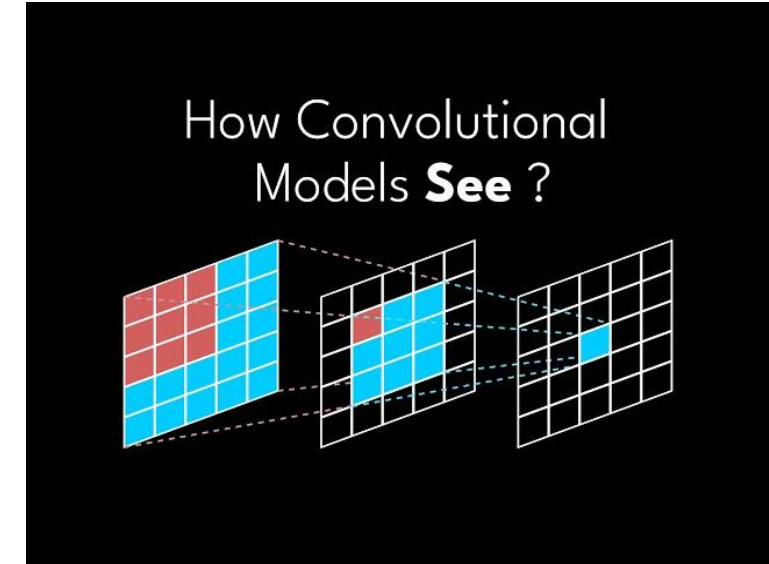
$$O = \frac{I - F + 2P}{S} + 1$$



1.2 Deep dive in CNN

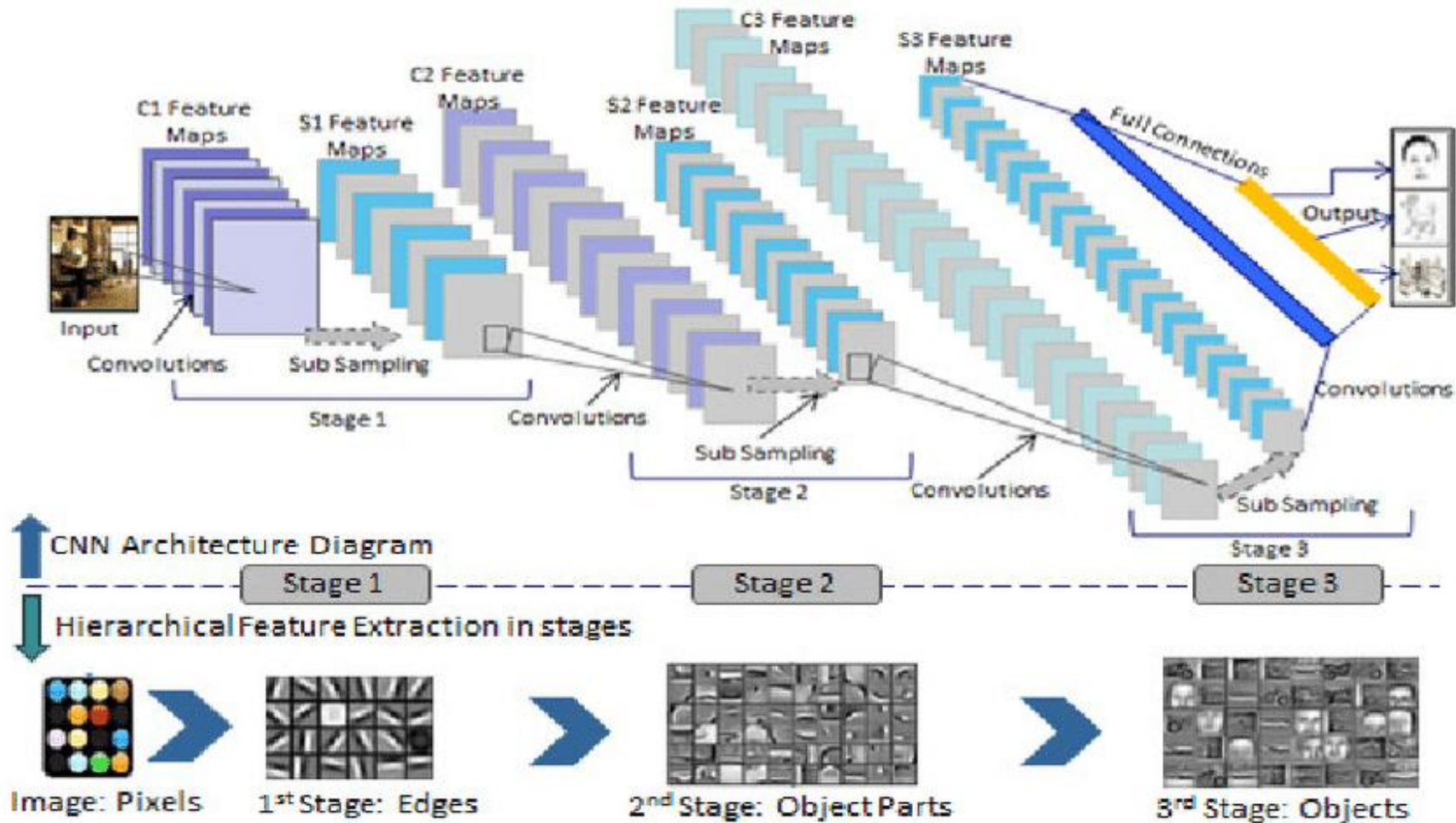
CNN의 특징 - receptive field

- Shallow Layers
 - 작은 receptive field → 픽셀 단위의 국소적인 특징 감지
 - 선, 모서리, 텍스처 같은 기초적인 기하학적 패턴 추출
- Deep Layers
 - Receptive field가 점점 커짐 → 더 넓은 영역에서 정보를 통합
 - 작은 특징들을 조합하여 고차원적인 의미론적 정보(semantic feature) 학습
 - 개별 선이나 패턴이 아니라, 사람, 동물, 사물의 형태와 의미를 인식
- receptive field
 - Conv filter의 크기가 커지면서 가지는 이점은 하나의 filter가 연산될 때 고려되는 input의 크기가 커진다
 - feature map을 얻기 위해 고려할 수 있는 입력의 spatial dim, 하나의 kernel이 볼 수 있는 이미지 레벨에서의 영역
- Alexnet → VGG : 11×11, 7×7, 5×5 보다 receptive field를 늘리는 차원에서는 3×3을 여러 번 사용하는 게 좋다



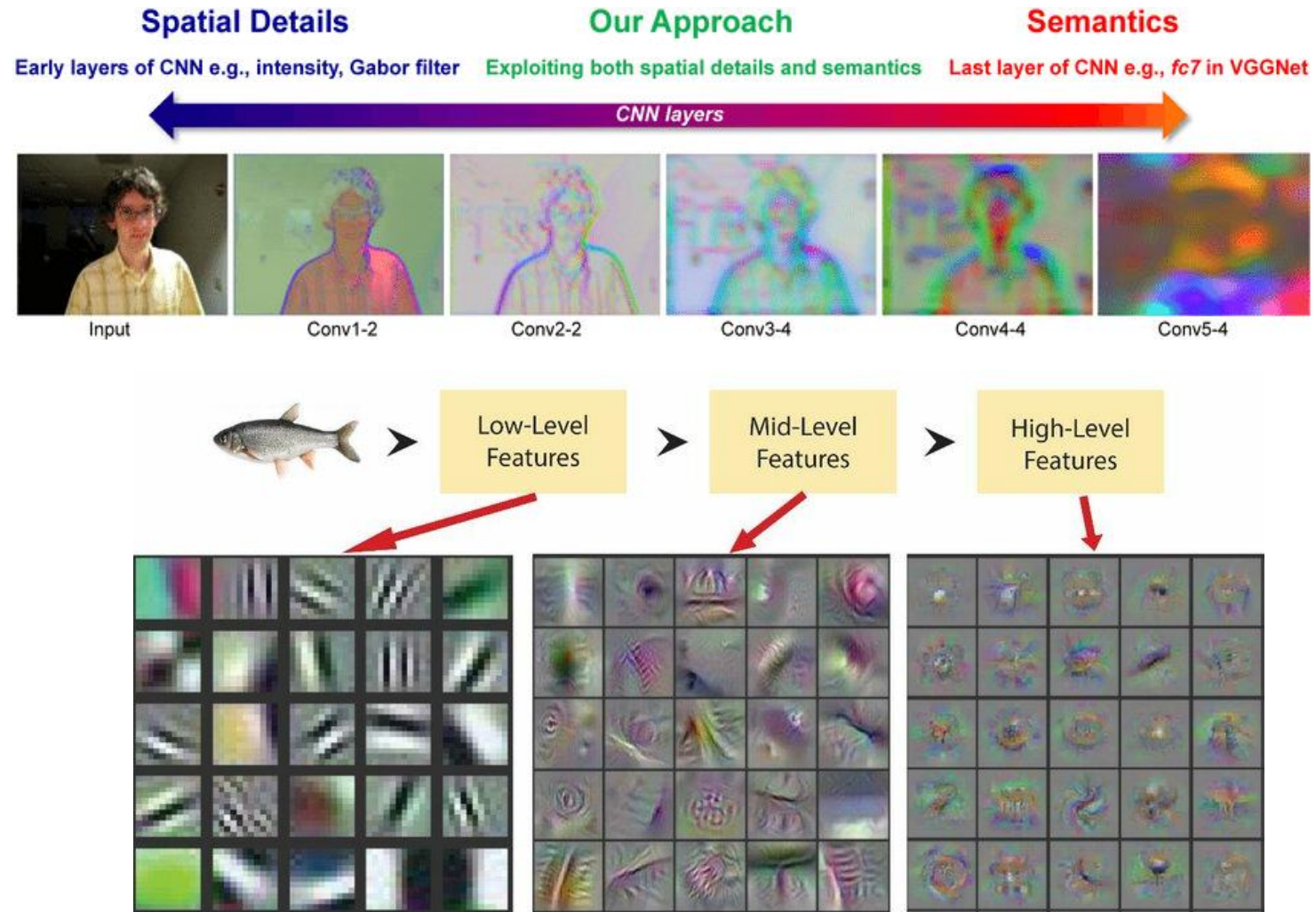
1.2 Deep dive in CNN

CNN의 특징 - hierarchical feature extraction



1.2 Deep dive in CNN

CNN의 특징 - hierarchical feature extraction



1.2 Deep dive in CNN

■ CNN의 특징 - hierarchical feature extraction : 무조건 layer를 깊게 쌓는 것이 좋은 게 아니다!

- 실제 문제를 해결할 때 해결해야 하는 문제에 따라 일부러 모델을 shallow하게 해서 선, 모서리 등의 low-level features만 활용하는 경우도 있음

1. 문서 이미지 처리 (OCR, 필기체 인식)

- 글자는 기본적으로 선과 곡선의 조합 → 복잡한 semantic feature보다 edge, corner 정보가 더 중요
- 너무 깊은 모델을 쓰면 불필요한 변형을 학습할 수도 있음

2. Defect Detection (결함 검사)

- 기계 부품, 반도체 웨이퍼, 의료 영상에서 스크래치, 크랙, 결함 같은 지역적 특징 탐지
- 결함 패턴이 특정 edge나 모양으로 나타나기 때문에 깊은 모델보다 shallow CNN으로 처리하는 게 효과적일 때도 있음

3. 얼굴 감지 (Face Detection)

- 얼굴을 인식하는 게 아니라, 얼굴이 있는지 여부만 판단할 경우
- 선, 경계, 윤곽 정보만 활용하는 Haar-like feature(흑백 이미지에서 밝기 차이(contrast)를 이용해 특징을 추출) + Shallow CNN이 빠르고 가벼움 (ex. Viola-Jones Face Detector)

Chapter II

Why is AlexNet important?

2.1 Why is AlexNet important?

ILSVRC

- ImageNet Large-Scale Visual Recognition Challenge
 - Classification / Detection / Localization / Segmentation
 - 1,000 different categories
 - Over 1 million images
 - Training set: 456,567 images

Image classification

Easiest classes



Hardest classes



2.1 Why is AlexNet important?

CNN의 역사적 흐름

- CNN은 1980년대부터 존재했지만, 데이터 부족 + 하드웨어 성능 한계로 널리 쓰이지 못 했었음. AlexNet이 나오면서 CNN이 본격적으로 이미지 인식 분야를 혁신

딥러닝의 부활 & 기존 머신러닝 방식과의 차이

- 2012년 ILSVRC (ImageNet Large Scale Visual Recognition Challenge)에서 기존 SVM, Random Forest 기반 전통적 머신러닝 모델들을 크게 압도
- 이후 VGG, GoogLeNet, ResNet 등 더 깊은 CNN 모델들이 등장하는 계기

하드웨어 (GPU) 활용 본격화

- AlexNet은 CNN 연산을 가속하기 위해 GPU (NVIDIA GTX 580)를 적극 활용한 최초의 모델
- 이후 GPU 최적화 연구가 본격적으로 진행되면서 딥러닝 프레임워크(TensorFlow, PyTorch) 발전의 계기가 됨

Key ideas

- AlexNet이 도입한 핵심 기술들은 이후 CNN 발전의 초석이 됨
- ReLU, Dropout, Overlapping Max Pooling, Data Augmentation

결론: AlexNet이 없었으면 지금의 딥러닝도 없었다

- AlexNet은 단순히 성능이 좋은 모델이 아니라, 딥러닝 시대를 연 역사적인 모델
- 이전에는 CNN이 제한적으로 사용되었지만, AlexNet 이후 딥러닝이 이미지 인식의 표준이 되었음

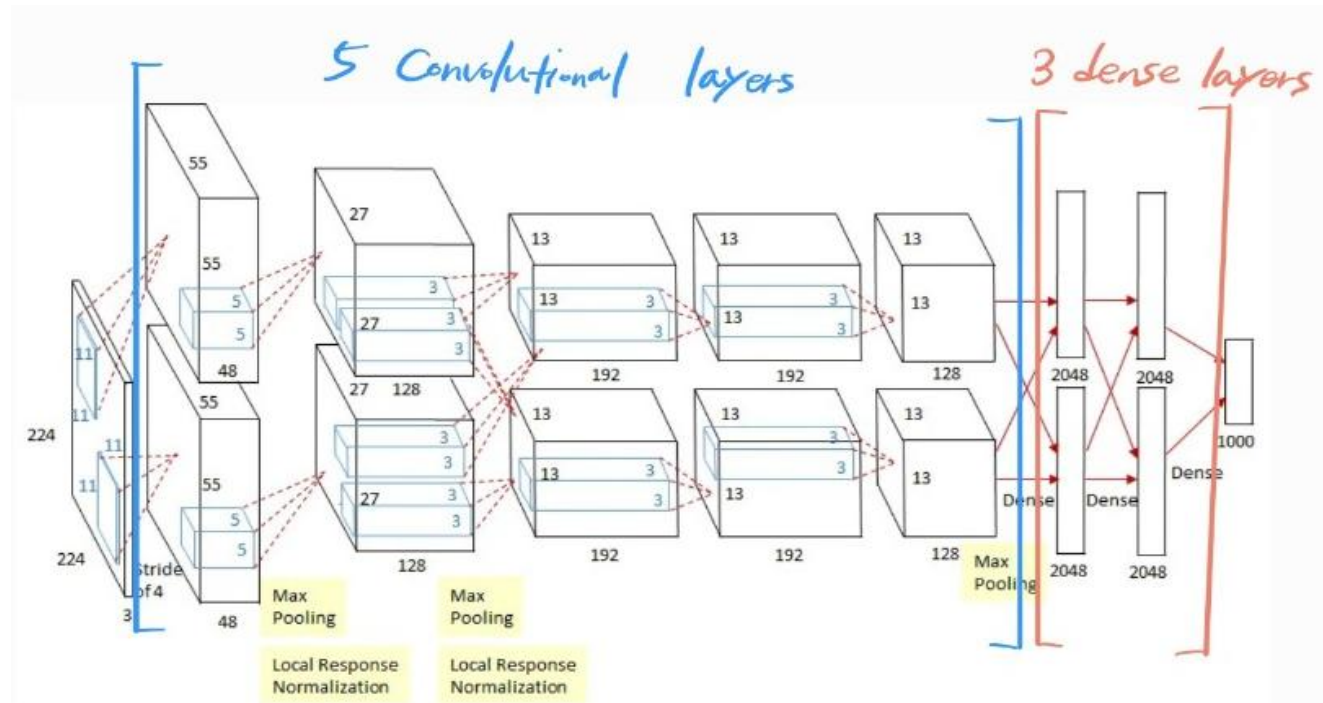
Chapter III

AlexNet - Architecture

3.1 AlexNet 구조

AlexNet 구조

- 네트워크가 2개의 path로 갈라짐 → 당시 사용하던 gpu 메모리가 크지 않아서 모든 파라미터가 한 번에 네트워크가 들어가지 않기 때문에 2개의 gpu로 나눠서 training
- Input에 11x11 filter를 사용함 → 파라미터 수 관점에서는 11x11은 좋은 선택이 아님, receptive field는 커지만 상대적으로 더 많은 파라미터 필요
- 총 depth는 8 (8 layers) → 요즘의 모델과 비교하면 많이 shallow 하지만 당시 2012에는 deep한 네트워크
- 1000개 카테고리의 데이터셋을 활용했기 때문에 마지막 1000개 노드
- 원래 논문에서는 입력 이미지 크기가 224×224×3으로 되어있지만, 실제 코드 구현에서는 padding을 적용하여 입력 크기를 227×227×3으로 변경



3.1 AlexNet 구조

AlexNet 구조 - 입력 이미지 전처리

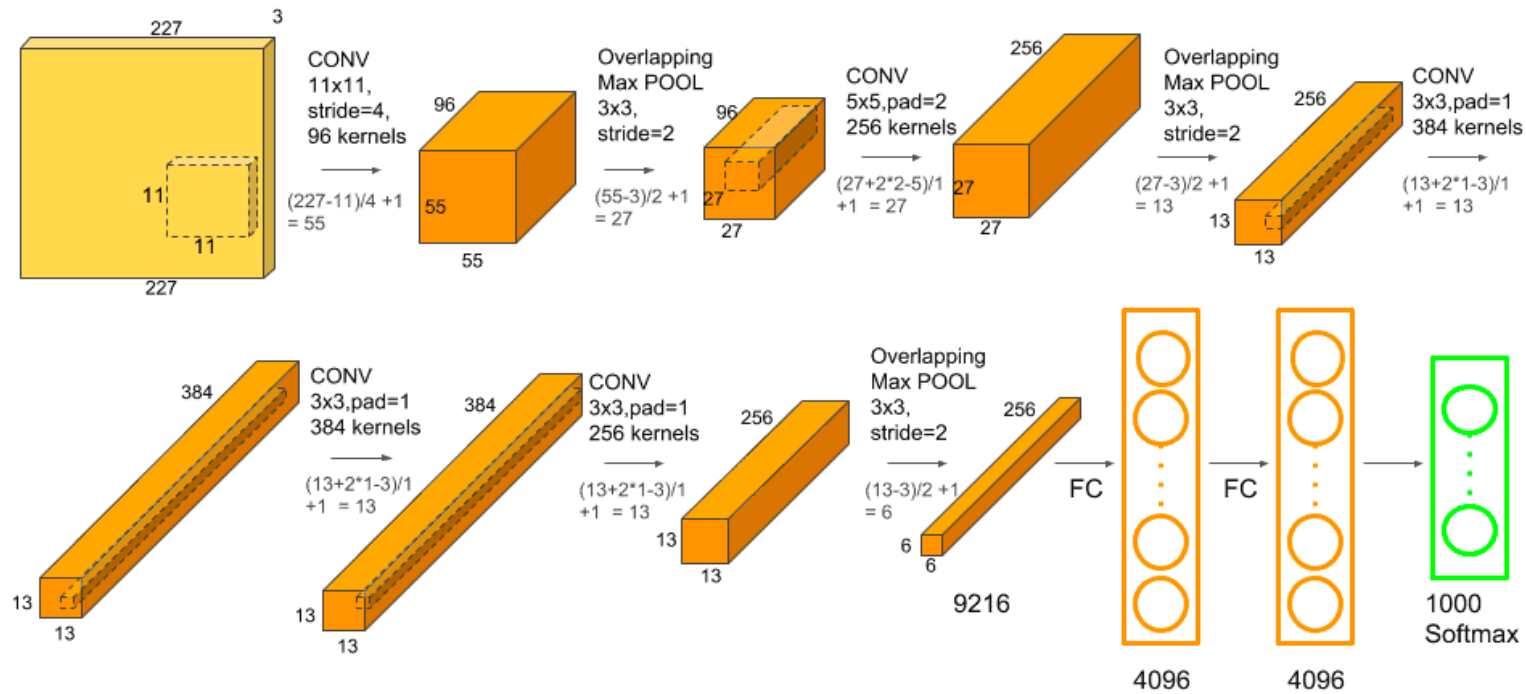
- 입력 이미지 크기는 227x227x3(RGB) 입니다. 원본 이미지 크기가 다르면 256x256로 리사이징한 후 227x227 영역을 잘라(crop)냄



3.1 AlexNet 구조

AlexNet 구조 - 입력 이미지 전처리

- 입력부터 완전 연결 계층 이전까지, 동일한 모델 구조를 두 가지로 병렬하여 빌드



3.1 AlexNet 구조

AlexNet 구조 - 입력 이미지 전처리

- 합성곱 계층 1
 - 11x11x3 크기의 커널 48개를 2개 그룹 사용하여, 55x55x(48x2) 특징맵 출력 (stride=4, padding=0)
 - 3x3 중첩 최대풀링 Overlapping Max Pooling 을 적용하여, 27x27x(48x2) 특징맵 출력 (stride=2)
 - 지역 응답 정규화 Local Response Normalization 를 적용하고, 27x27x(48x2) 특징맵을 출력
- 합성곱 계층 2
 - 5x5x48 크기의 커널 128개를 2개 그룹 사용하여, 27x27x(128x2) 특징맵 출력 (stride=1, padding=2)
 - 3x3 중첩 최대풀링 적용하여, 13x13x128x2 특징맵 출력 (stride=2)
 - 지역 응답 정규화를 적용하고, 13x13x128x2 특징맵 출력
- 합성곱 계층 3
 - 3x3x256 크기의 커널 192개를 2개 그룹 사용하여, 13x13x192x2 특징맵 출력 (stride=1, padding=1)
- 합성곱 계층 4
 - 3x3x192 크기의 커널 192개를 2개 그룹 사용하여, 13x13x192x2 특징맵 출력 (stride=1, padding=1)
- 합성곱 계층 5
 - 3x3x192 크기의 커널 128개를 2개 그룹 사용하여, 13x13x128x2 특징맵 출력 (stride=1, padding=1)
 - 3x3 중첩 최대풀링(stride=2)을 적용하여, 6x6x128x2 특징맵 출력
- 완전연결 계층 6-8
 - 5계층 합성곱 후 6계층, 7계층에서는 각각 4096개의 노드
 - 마지막 8계층에는 1000개의 노드가 있어 1000개 클래스 구분을 의미하는 확률값을 출력(softmax)
- 파라미터 수와 계산 복잡도
 - AlexNet에는 약 6천만 개의 학습 가능한 파라미터가 있습니다. 당시 GPU 환경에서 5~6일(!)이 걸렸던 대규모 모델

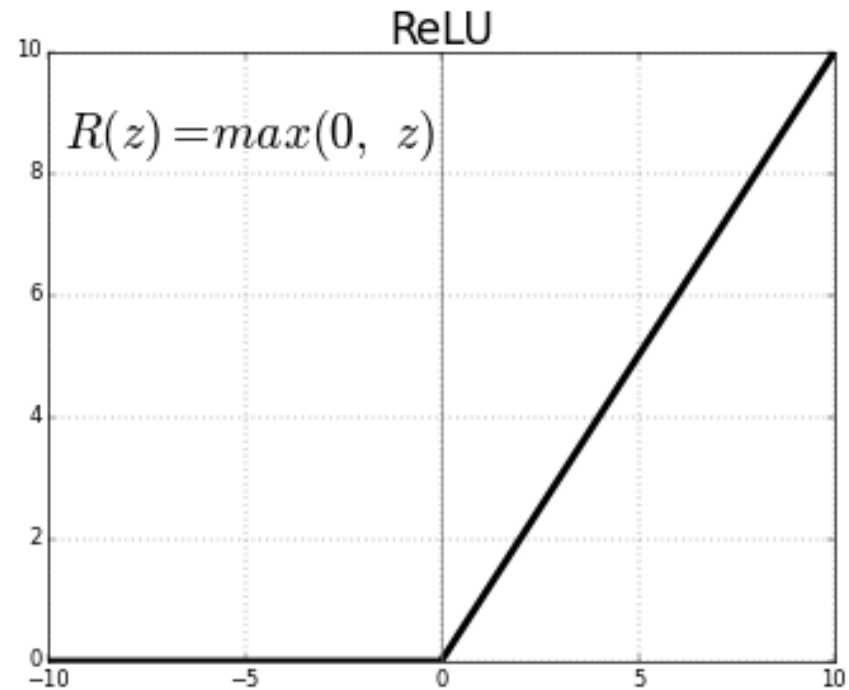
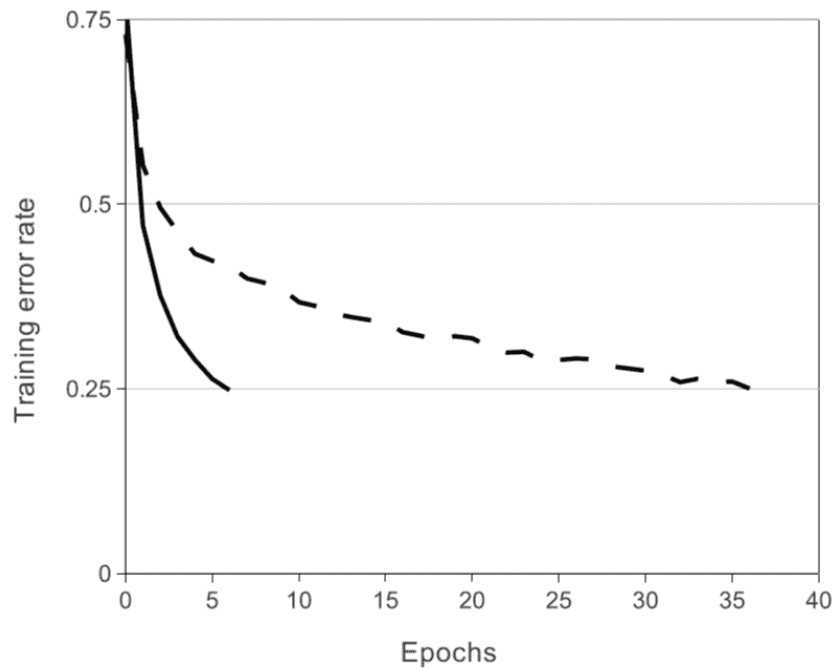
Chapter IV

AlexNet - Key ideas

4.1 AlexNet - Key ideas

Rectified Linear Unit (ReLU) activation

- AlexNet using ReLUs(solid curve) could achieve a 25% training error rate six times faster than an equivalent network using tanh(dotted curve)
- Vanishing gradient가 해결되기 때문



4.1 AlexNet - Key ideas

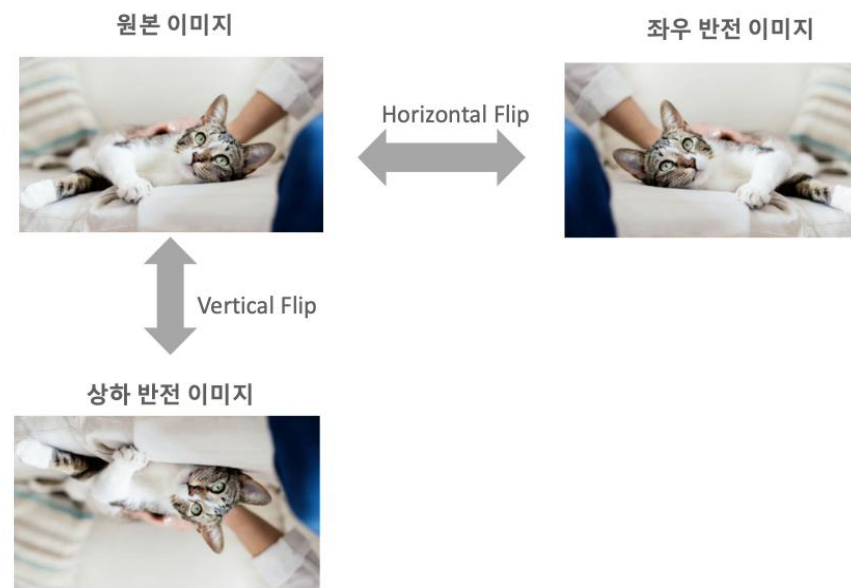
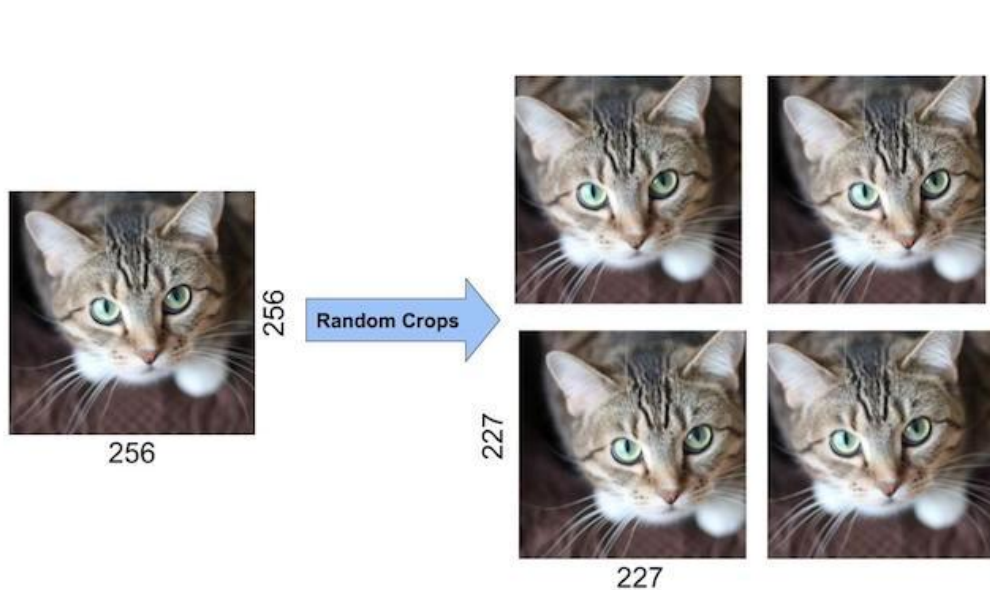
GPU implementation (2 GPUs)

- 단일 GTX 580 GPU는 메모리가 3GB에 불과하여 학습할 수 있는 네트워크의 최대 크기에 제한
- 따라서 우리는 네트워크를 두 개의 GPU에 분산하여 학습
- 동일한 합성곱 계층 구조가 두 개의 그룹으로 설정되어 각 GPU에서 계산

4.1 AlexNet - Key ideas

Data augmentation

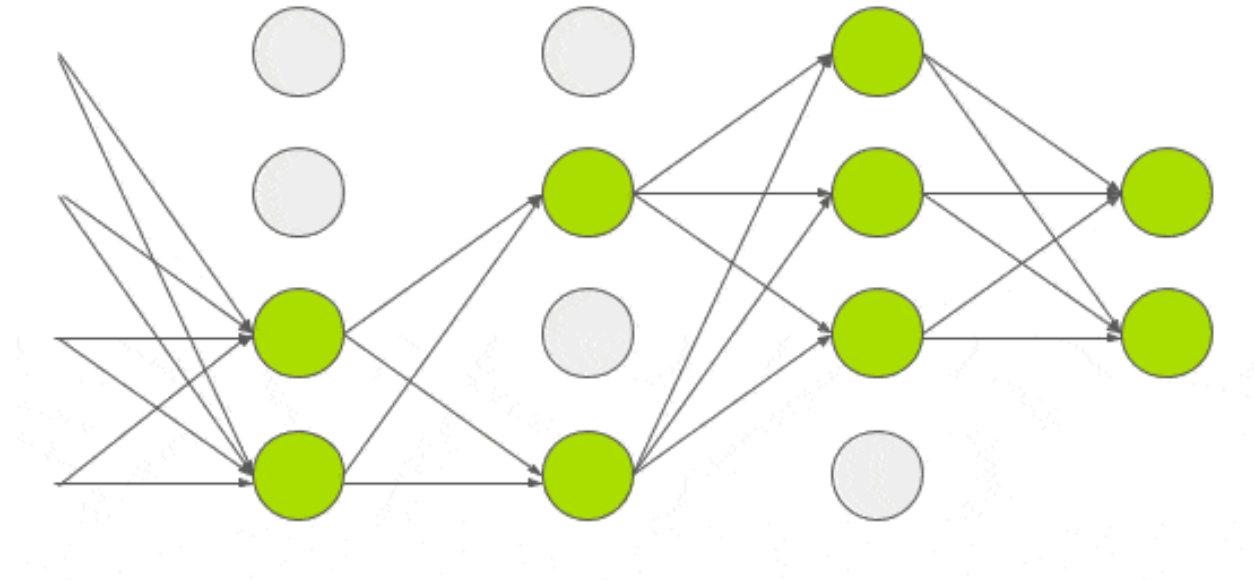
- AlexNet에서는 데이터 증강 기법을 활용해 총 2048 배의 데이터셋을 만들어내면서, 효과적으로 오버피팅 문제를 해결
- 이미지 평행 이동: 입력 이미지에서 무작위로 227×227 영역을 잘라내어 데이터 수를 1024배 증가
- 수평 반전(**Horizontal Flip**): 이미지를 수평으로 반전시켜 데이터 수를 2배로 늘림
- 조명 변화: PCA 기법을 활용해 각 이미지의 픽셀 강도를 부드럽게 변형



4.1 AlexNet - Key ideas

Dropout

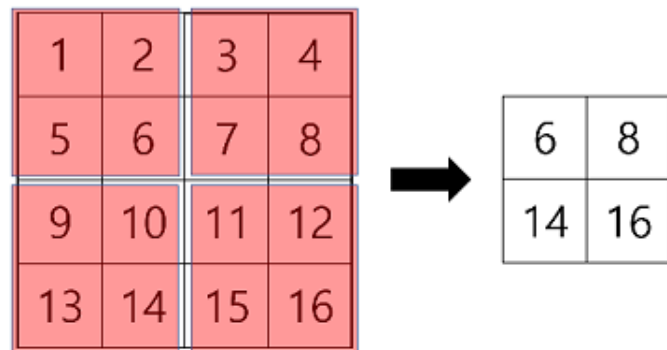
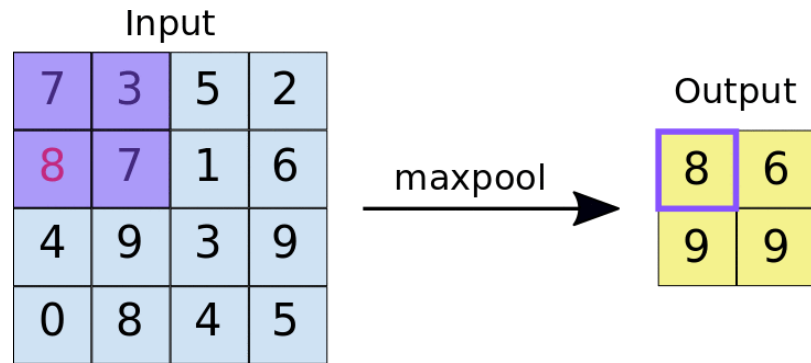
- Dropout은 신경망의 과적합을 방지하기 위한 regularization 기술
- 각 iteration(데이터 1개) 마다 은닉층의 일부 노드를 무작위로 꺼서 과적합을 방지
- AlexNet에서는 완전연결 계층에 0.5 확률의 dropout을 적용
- 테스트 시에는 모든 뉴런을 활성화하지만, 출력값을 0.5배로 조정



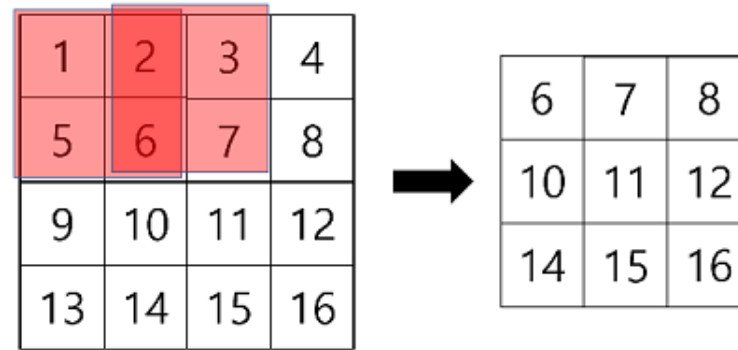
4.1 AlexNet - Key ideas

Overlapping Max pooling

- 일반적인 Max pooling은 커널 크기와 스트라이드 크기가 동일. 하지만 AlexNet에서는 3×3 크기 커널을 2픽셀 단위로 움직이며 pooling을 수행하는 Overlapping Max pooling을 사용
- 이를 통해 Top-1 오차율 0.4%, Top-5 오차율 0.3%가 감소하는 효과가 있었음. 중첩 pooling이 더 세밀한 특징을 추출할 수 있었기 때문



Non-Overlapping Pooling



Overlapping Pooling

4.1 AlexNet - Key ideas

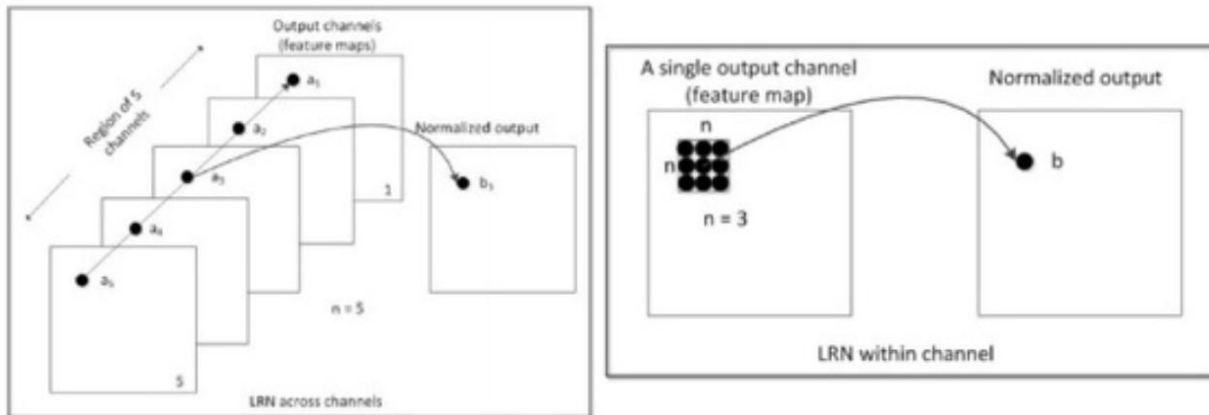
Local response normalization

- AlexNet에서는 합성곱 계층 후에 지역 응답 정규화 계층을 추가했습니다. 이 계층은 입력 매핑(feature map) 각각의 뉴런에 대해 주변 뉴런들의 활성화 정도에 의해 정규화를 수행
- 생물학적 신경망에서 관찰되는 "Lateral Inhibition(측방 억제)" 현상에서 영감을 받았습니다. 즉, 강한 자극을 받은 뉴런이 주변 뉴런의 활성화를 억제하는 방식
- Alexnet은 왜 LRN을 사용? → ReLU 때문
- ReLU는 양수에서는 입력 값을 그대로 사용. 그럼 conv, pooling 시 매우 높은 하나의 픽셀 값이 주변에 영향
- LRN은 큰 값을 가진 뉴런의 영향을 감소시켜 학습 안정성을 높임
- 지역 응답 정규화를 통해 Top-1, Top-5 오차율이 각각 1.4%, 1.2% 감소하는 효과. 이는 학습 수렴 속도를 높여주었기 때문. 현재는 대신 **배치 정규화(Batch Normalization)**가 주로 사용

4.1 AlexNet - Key ideas

Local response normalization

- Tries to mimic the inhibition scheme in the brain



LRN은 주어진 feature map의 특정 위치에서 특정 채널의 값 $a_{x,y}^i$ 을 정규화하는 방식으로 작동합니다.

$$b_{x,y}^i = \frac{a_{x,y}^i}{\left(k + \alpha \sum_{j=\max(0, i-\frac{N}{2})}^{\min(C-1, i+\frac{N}{2})} (a_{x,y}^j)^2\right)^\beta}$$

여기서:

- $a_{x,y}^i$: (x, y) 위치에서 채널 i 의 활성값 (입력 값)
- $b_{x,y}^i$: 정규화된 출력 값
- k : 작은 양수(통상적으로 2), 분모가 0이 되는 것을 방지
- α : scaling factor (일반적으로 0.0001)
- β : normalization exponent (일반적으로 0.75)
- N : normalization window 크기 (보통 5)
- C : 전체 채널 개수

Reference

Reference

- Fei-FeiLi, Justin Johnson, Serena Yeung, CS231n
- <https://legacy.gitbook.com/@leonardoaraujosantos>
- 네이버 부스트코스 - 인공지능 기초 다지기 (AI Basic) 강의
- https://github.com/vdumoulin/conv_arithmetic
- output feature map 계산 공식 : <https://woochan-autobiography.tistory.com/884#2.%20Stride%EA%B0%80%201%EC%9D%B4%EA%B3%A0%20Padding%EC%9D%B4%20%EC%97%86%EB%8A%94%20%EA%B2%BD%EC%9A%B0>
- Alexnet : https://modulabs.co.kr/blog/alexnet_structure